

Conceitos básicos

Estruturas de Dados

Prof. Dr. Franklin César Flores

Departamento de Informática

Universidade Estadual de Maringá

Conceitos básicos

- Este conjunto de slides foi utilizado durante as aulas das disciplinas:
- 6884 – Estruturas de Dados – Turma 02 (Ciência da Computação)
- 9893 – Estruturas de Dados – Turma 31 (Informática)
- Maringá, 2º semestre letivo de 2024

Roteiro

- Algoritmos e estruturas de dados
 - Tipo Abstrato de Dados
- Noções de complexidade de algoritmos
 - Revisão sobre recursão
- Formas de alocação
 - Revisão sobre ponteiros
- Exercícios
- Bibliografia

Algoritmos e estruturas de dados

- Algoritmo
 - Um algoritmo é uma sequência finita, lógica e não ambígua de instruções computacionais que transformam um conjunto de valores de entrada em um conjunto de valores de saída
 - É uma sequência de ações executáveis para a solução de um problema computacional específico
 - Um programa é uma sequência de instruções a serem executadas por um computador
 - Nesta disciplina, utilizaremos um pseudocódigo para escrever os algoritmos que serão estudados

Algoritmos e estruturas de dados

- Tipo de dado
 - É a caracterização de um conjunto de valores associados a constantes, variáveis, expressões ou saídas de funções
 - O tipo de dado define características tais como o conjunto de valores que podem ser representados e as operações que podem ser realizadas com eles
 - Os tipos de dados podem ser:
 - Simples ou primitivo: são grupos de valores indivisíveis (ex: inteiro, real, caractere, booleano)
 - Composto ou estruturado: são coleções de valores primitivos ou agregados de valores de tipos diferentes

Algoritmos e estruturas de dados

- Estrutura de dados
 - São estruturas que armazenam e organizam dados objetivando acesso e modificações facilitados
 - É uma coleção organizada de valores de dados bem como das operações de gerenciamento desta estrutura
 - Estruturas de dados requerem **algoritmos** para sua criação e manipulação
 - Podem ser construídas a partir de tipos de dados primitivos
 - O projeto de estruturas de dados leva em conta critérios como armazenamento e as operações de gerenciamento
 - São representações concretas de tipos abstratos de dados sob o ponto de vista de **implementador**

Algoritmos e estruturas de dados

- Tipo Abstrato de Dados
 - Um Tipo Abstrato de Dados (TAD) é um modelo matemático para um conjunto de dados. À este modelo, está associado um conjunto de operações de gerenciamento
 - A definição de um TAD permite a abstração de como o mesmo foi implementado. Pode-se, assim, se concentrar somente na natureza do tipo de dado
 - Um TAD pode ser definido por sua semântica sob o ponto de vista de **usuário**

Algoritmos e estruturas de dados

- Exemplos de Tipos Abstratos de Dados
 - Pilha: uma pilha é uma estrutura de armazenamento de dados que admite inserções, remoções e acessos somente em relação ao topo da mesma
 - Árvore: uma árvore é uma estrutura que, ou é vazia ou é composta por um nó que possui um conjunto de árvores descendentes. Operações de inserção, remoção e busca podem ser realizadas na estrutura

Algoritmos e estruturas de dados

- Diferença entre Estruturas de Dados e Tipos Abstratos de Dados
 - Um TAD pode ser definido por sua semântica sob o ponto de vista de **usuário**
 - Pode-se definir e utilizar TADs sem se conhecer nenhum detalhe sobre sua implementação
 - Por outro lado, estruturas de dados são representações concretas de tipos abstratos de dados sob o ponto de vista de **implementador**
 - Um TAD admite mais de um tipo de implementação

Algoritmos e estruturas de dados

Usuário

leia(x);

InicializaFila(Q);

Processa(x);

InserereFila(Q, x);

enquanto NÃO FilaVazia(Q)

faça RemoveFila(Q, x);

para cada viz vizinho de x

faça se NÃO Processado(viz)

então Processa(viz);

 InserereFila(Q, viz);

Implementador

InicializaFila(Q);

 Q.Início $\leftarrow$ -1;

 Q.Fim $\leftarrow$ Q.Início;

FilaVazia(Q);

 FilaVazia $\leftarrow$ Q.Início = Q.Fim;

FilaCheia(Q);

 FilaCheia $\leftarrow$ (Q.Início = -1) $\underline{\text{E}}$ (Q.Fim = TamMax);

InserereFila(Q, x);

se NÃO FilaCheia(Q)

então se Q.Fim = TamMax

então DeslocaFila(Q);

 Q.Fim $\leftarrow$ Q.Fim + 1;

 Q.Elem[Q.Fim] $\leftarrow$ x;

RemoveFila(Q, x);

se NÃO FilaVazia(Q)

então Q.Início $\leftarrow$ Q.Início + 1;

 x $\leftarrow$ Q.Elem[Q.Início];

Complexidade de algoritmos

- A análise de um algoritmo consiste na avaliação dos recursos necessários para que o mesmo seja executado.
- Requerimentos de tempo e de espaço são exemplos de recursos que podem ser avaliados
- Tempo : quanto tempo é necessário para executar um algoritmo dado um problema de tamanho n ?
- Espaço : qual a quantidade de memória requerida pela execução de um algoritmo quando o tamanho do problema é n ?
- Nesta disciplina, o nosso foco estará na avaliação do **tempo**

Complexidade de algoritmos

- A análise de algoritmos permite que sejam identificados os algoritmos mais eficientes dentro de uma classe de algoritmos aplicáveis para a resolução de um determinado problema.
- Dois exemplos de situações onde a análise de algoritmos pode ser aplicada:
 - Qual o custo para resolver um dado problema utilizando **um algoritmo específico**?
 - Dentro de uma **classe de algoritmos**, qual o algoritmo de menor custo para resolver um determinado problema?

Complexidade de algoritmos

- Algumas maneiras para se medir o custo de um algoritmo:
- Execução de um programa em um computador real
 - Medida direta do tempo de execução (não é a mais adequada)
 - Dependência da tecnologia aplicada
- Utilização de um modelo matemático baseado em um computador idealizado
 - As operações são especificadas e custos são atribuídos a elas
 - É mais usual que sejam ignorados os custos das operações, bem com algumas das operações menos significativas

Complexidade de algoritmos

- Para medir o custo de execução de um algoritmo, deve-se determinar para o mesmo uma ou mais funções de complexidade
- Estas funções relacionam o tamanho da entrada de um problema (n) com a quantidade de recursos necessários
- Uma função de complexidade $T(n)$ é uma **função de complexidade de tempo** ou **função de tempo de execução** se ela mede a quantidade de tempo necessário para executar um algoritmo com uma entrada de tamanho n
- Uma função de complexidade $E(n)$ é uma **função de complexidade de espaço** se ela medir a quantidade de memória necessária para executar um algoritmo com uma entrada de tamanho n

Complexidade de algoritmos

- Dado o conjunto de todas as possíveis entradas de tamanho n para um determinado algoritmo, três cenários são de muito interesse:
- O **melhor caso** corresponde ao menor uso do recurso avaliado sobre todas as possíveis entradas de tamanho n
- O **pior caso** corresponde ao maior uso do recurso avaliado sobre todas as possíveis entradas de tamanho n
- O **caso médio** corresponde à média dos recursos utilizados para todas as entradas de tamanho n

Complexidade de algoritmos

- Como o **tempo de execução** será o foco nesta disciplina, vale salientar:
- O **melhor caso** corresponde ao menor tempo de execução sobre todas as possíveis entradas de tamanho n
- O **pior caso** corresponde ao maior tempo de execução sobre todas as possíveis entradas de tamanho n
- O **caso médio** corresponde à média dos tempos de execução de todas as entradas de tamanho n
- A complexidade de tempo não está relacionada ao tempo em si mas **ao número de vezes em que as instruções do algoritmo são executadas**

Complexidade de algoritmos

- A **função de tempo de execução** $T(n)$ é dada pelo **custo** de cada instrução e pelo **número de vezes** que cada instrução é executada
- Considere que c_i é o custo da instrução na i -ésima linha do algoritmo
- O custo da execução de uma linha do algoritmo se dá pelo produto $c_i \times$ (número de vezes que a linha i é executada)
- $T(n)$ é computada pela somatória dos produtos de todas as linhas

Complexidade de algoritmos

- Função crescente:
- Uma função f é crescente se $x \leq y \Rightarrow f(x) \leq f(y)$.
- Todas função de tempo de execução $T(n)$ é uma função crescente
- Quanto maior o valor de n , maior o valor de $T(n)$. Ou seja, **$T(n)$ cresce à medida que n cresce**
 - Quanto maior o tamanho do problema, maior o tempo necessário para resolver o problema
 - Exceção: existem algoritmos cujo tempo de execução é constante
- Estudaremos crescimento de funções em breve

Complexidade de algoritmos

	<u>custo</u>	<u>nº execuções</u>
<u>leia</u> (a);		
b ← a + 1;		
<u>escreva</u> (b);		

$T(n) =$

Complexidade de algoritmos

	<u>custo</u>	<u>nº execuções</u>
<u>leia</u> (a);	c_1	1
$b \leftarrow a + 1;$	c_2	1
<u>escreva</u> (b);	c_3	1

$$T(n) = c_1 + c_2 + c_3$$

Complexidade de algoritmos

- A **função de tempo de execução** $T(n)$ é dada pelo **custo** de cada instrução e pelo **número de vezes** que cada instrução é executada
- Como dito anteriormente, pode-se também ignorar os custos individuais de cada instrução e computar $T(n)$ simplesmente pela **somatória do número de execuções de cada linha** do algoritmo
- Esta será a maneira que definiremos as funções $T(n)$ ao longo da disciplina

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (a);	
b ← a + 1;	
<u>escreva</u> (b);	

$T(n) =$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (a);	1
b ← a + 1;	1
<u>escreva</u> (b);	1

$$T(n) = 3$$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (a);	
<u>se</u> a > 0	
<u>então</u> a ← 2 * a;	
b ← a + 100;	
<u>escreva</u> (a);	

$T(n) =$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (a);	1
<u>se</u> a > 0	1
<u>então</u> a ← 2 * a;	0 <u>ou</u> 1
b ← a + 100;	0 <u>ou</u> 1
<u>escreva</u> (a);	1

$$T(n) = \begin{cases} 5, & \text{se } a > 0 \\ 3, & \text{caso contrário} \end{cases}$$

Complexidade de algoritmos

<u>nº execuções</u>	
<u>leia</u> (a);	
<u>se</u> a > 0	
<u>então</u>	a ← 2 * a;
	b ← a + 100;
<u>senão</u>	a ← 3 * a;
	c ← 50;
	b ← a - c;
<u>escreva</u> (b);	

$T(n) =$

Complexidade de algoritmos

		<u>nº execuções</u>
<u>leia</u> (a);		1
<u>se</u> a > 0		1
<u>então</u>	a ← 2 * a;	0 <u>ou</u> 1
	b ← a + 100;	0 <u>ou</u> 1
<u>senão</u>	a ← 3 * a;	1 <u>ou</u> 0
	c ← 50;	1 <u>ou</u> 0
	b ← a - c;	1 <u>ou</u> 0
<u>escreva</u> (b);		1

$$T(n) = \begin{cases} 5, & \text{se } a > 0 \\ 6, & \text{caso contrário} \end{cases}$$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	
i ← 1;	
<u>enquanto</u> i <= n	
<u>faça</u> <u>escreva</u> (i);	
i ← i + 1;	

$$T(n) =$$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	1
i ← 1;	1
<u>enquanto</u> i <= n	n + 1
<u>faça</u> <u>escreva</u> (i);	n
i ← i + 1;	n

$$T(n) = 3n + 3$$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	
i ← 0;	
<u>enquanto</u> i <= n	
<u>faça</u> <u>escreva</u> (i);	
i ← i + 1;	

$T(n) =$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	1
i ← 0;	1
<u>enquanto</u> i <= n	n + 2
<u>faça</u> <u>escreva</u> (i);	n + 1
i ← i + 1;	n + 1

$$T(n) = 3n + 6$$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	
i ← 0;	
<u>enquanto</u> i <= n	
<u>faça</u> <u>escreva</u> (i);	
i ← i + 2;	

$T(n) =$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	1
i ← 0;	1
<u>enquanto</u> i <= n	$\lfloor n/2 \rfloor + 2$
<u>faça</u> <u>escreva</u> (i);	$\lfloor n/2 \rfloor + 1$
i ← i + 2;	$\lfloor n/2 \rfloor + 1$

$$T(n) = 3\lfloor n/2 \rfloor + 6$$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	
i ← 1;	
<u>repita</u>	
<u>escreva</u> (i);	
i ← i + 1;	
<u>até</u> i > n;	

$T(n) =$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	1
i ← 1;	1
<u>repita</u>	
<u>escreva</u> (i);	n
i ← i + 1;	n
<u>até</u> i > n;	n

$$T(n) = 3n + 2$$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	
<u>para</u> i ← 1 <u>até</u> n	
<u>faça</u> <u>escreva</u> (i);	

$T(n) =$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	1
<u>para</u> i ← 1 <u>até</u> n	n + 1
<u>faça</u> <u>escreva</u> (i);	n

$$T(n) = 2n + 2$$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	
<u>para</u> i ← 0 <u>até</u> n <u>passo</u> 2	
<u>faça</u> <u>escreva</u> (i);	

$T(n) =$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	1
<u>para</u> i ← 0 <u>até</u> n <u>passo</u> 2	$\lfloor n/2 \rfloor + 2$
<u>faça</u> <u>escreva</u> (i);	$\lfloor n/2 \rfloor + 1$

$$T(n) = 2\lfloor n/2 \rfloor + 4$$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	
<u>para</u> i ← n <u>até</u> 1 <u>passo</u> -1	
<u>faça</u> <u>escreva</u> (i);	

$T(n) =$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	1
<u>para</u> i ← n <u>até</u> 1 <u>passo</u> -1	n + 1
<u>faça</u> <u>escreva</u> (i);	n

$$T(n) = 2n + 2$$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	
<u>para</u> i ← 1 <u>até</u> n	
<u>faça</u> <u>para</u> j ← 1 <u>até</u> n	
<u>faça</u> <u>escreva</u> (i, j);	

$$T(n) =$$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	1
<u>para</u> i ← 1 <u>até</u> n	n + 1
<u>faça</u> <u>para</u> j ← 1 <u>até</u> n	n · (n + 1)
<u>faça</u> <u>escreva</u> (i, j);	n · n

$$T(n) = 2n^2 + 2n + 2$$

Complexidade de algoritmos

<u>nº execuções</u>		
<u>leia</u> (n,a);		
<u>para</u> i ← 1 <u>até</u> n		
<u>faça</u> <u>se</u> a <> 0		
<u>então</u>	<u>para</u> j ← 1 <u>até</u> n	
	<u>faça</u> <u>escreva</u> (i, j);	

$T(n) =$

Complexidade de algoritmos

		<u>nº execuções</u>
<u>leia</u> (n,a);		1
<u>para</u> i ← 1 <u>até</u> n		n + 1
<u>faça</u> <u>se</u> a <> 0		n
<u>então</u> <u>para</u> j ← 1 <u>até</u> n		0 <u>ou</u> n · (n + 1)
<u>faça</u> <u>escreva</u> (i, j);		0 <u>ou</u> n · n

$$T(n) = \begin{cases} 2n^2 + 3n + 2, & \text{se } a \neq 0 \\ 2n + 2, & \text{caso contrário} \end{cases}$$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (a);	
<u>enquanto</u> a >= 0	
<u>faça</u> <u>escreva</u> (a);	
<u>leia</u> (a);	

$T(n) =$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (a);	1
<u>enquanto</u> a >= 0	j + 1
<u>faça</u> <u>escreva</u> (a);	j
<u>leia</u> (a);	j

$$T(n) = 3j + 2, \forall j \geq 0$$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	
i ← 1;	
<u>leia</u> (a);	
<u>enquanto</u> (a >= 0) <u>E</u> (i <= n)	
<u>faça</u> <u>escreva</u> (a);	
<u>leia</u> (a);	
i ← i + 1;	

$T(n) =$

Complexidade de algoritmos

	<u>nº execuções</u>
<u>leia</u> (n);	1
i ← 1;	1
<u>leia</u> (a);	1
<u>enquanto</u> (a >= 0) <u>E</u> (i <= n)	j + 1
<u>faça</u> <u>escreva</u> (a);	j
<u>leia</u> (a);	j
i ← i + 1;	j

$$T(n) = 4j + 4, \forall 0 \leq j \leq n$$

Complexidade de algoritmos

- Crescimento de funções
- Como vimos anteriormente, uma função f é crescente se $x \leq y \Rightarrow f(x) \leq f(y)$.
- Ou seja, dados dois valores de entrada x e y , se x for menor ou igual a y , então o valor de $f(x)$ será menor ou igual a $f(y)$
- As funções de tempo de execução $T(n)$ são funções crescentes
- Quanto maior o valor de n , maior o custo (tempo) para se resolver o problema com o algoritmo associado a $T(n)$ (exceto quando $T(n)$ for uma função constante)

Complexidade de algoritmos

- Comportamento assintótico de funções
- Na escolha de um algoritmo para um problema computacional específico, analisamos o impacto do tamanho n do problema na obtenção da solução por este algoritmo
- Quando n é **suficientemente pequeno**, pouco custam os algoritmos para serem executados e a escolha dentro de uma classe de algoritmos para o referido problema não é crítica

Complexidade de algoritmos

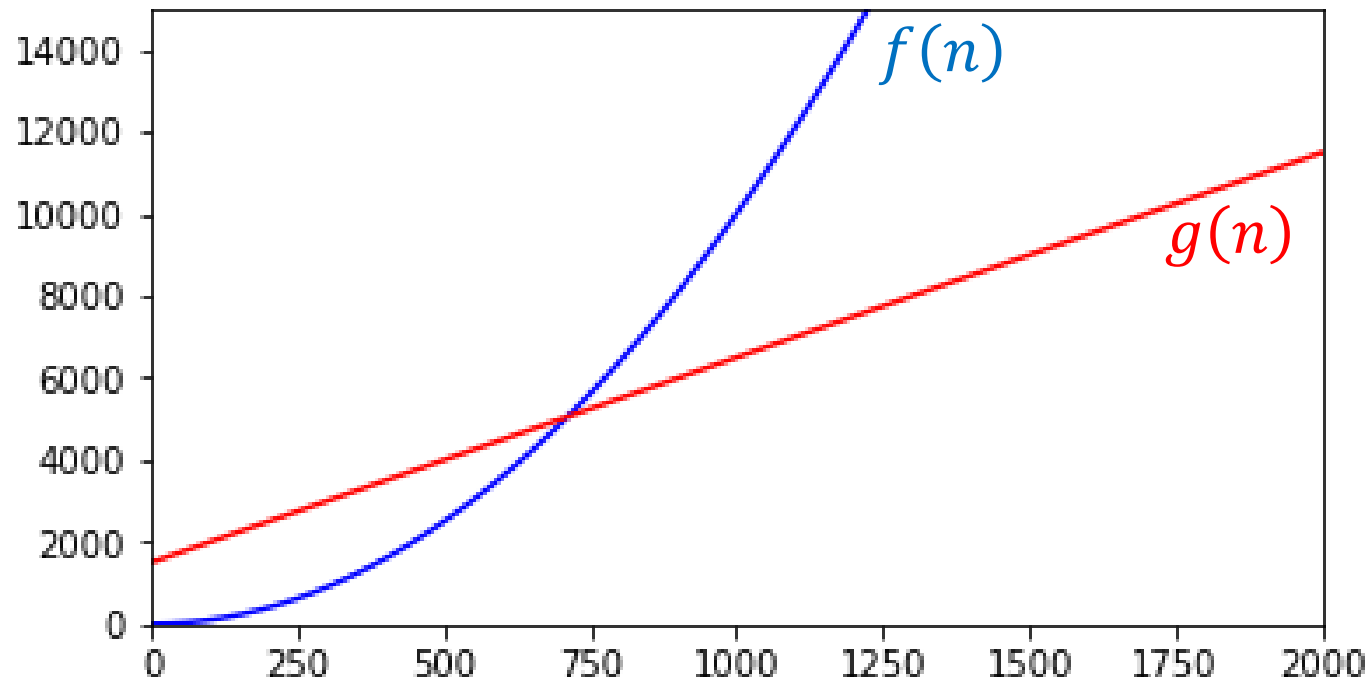
- Comportamento assintótico de funções
- Na escolha de um algoritmo para um problema computacional específico, analisamos o impacto do tamanho n do problema na obtenção da solução por este algoritmo
- Quando n é **suficientemente grande**, ou seja, para grandes valores de n , analisamos o **comportamento assintótico** das funções de tempo de execução $T(n)$

Complexidade de algoritmos

- Comportamento assintótico de funções
- Quando n é **suficientemente grande**, ou seja, para grandes valores de n , analisamos o **comportamento assintótico** das funções de tempo de execução $T(n)$
- A eficiência **assintótica** de um algoritmo é a maneira como $T(n)$ aumenta no **limite** à medida que o tamanho n aumenta indefinidamente
- Dentro de uma classe de algoritmos para um determinado problema computacional, escolhemos o algoritmo assintoticamente mais eficiente para n suficientemente grande

Complexidade de algoritmos

- Comportamento assintótico de funções

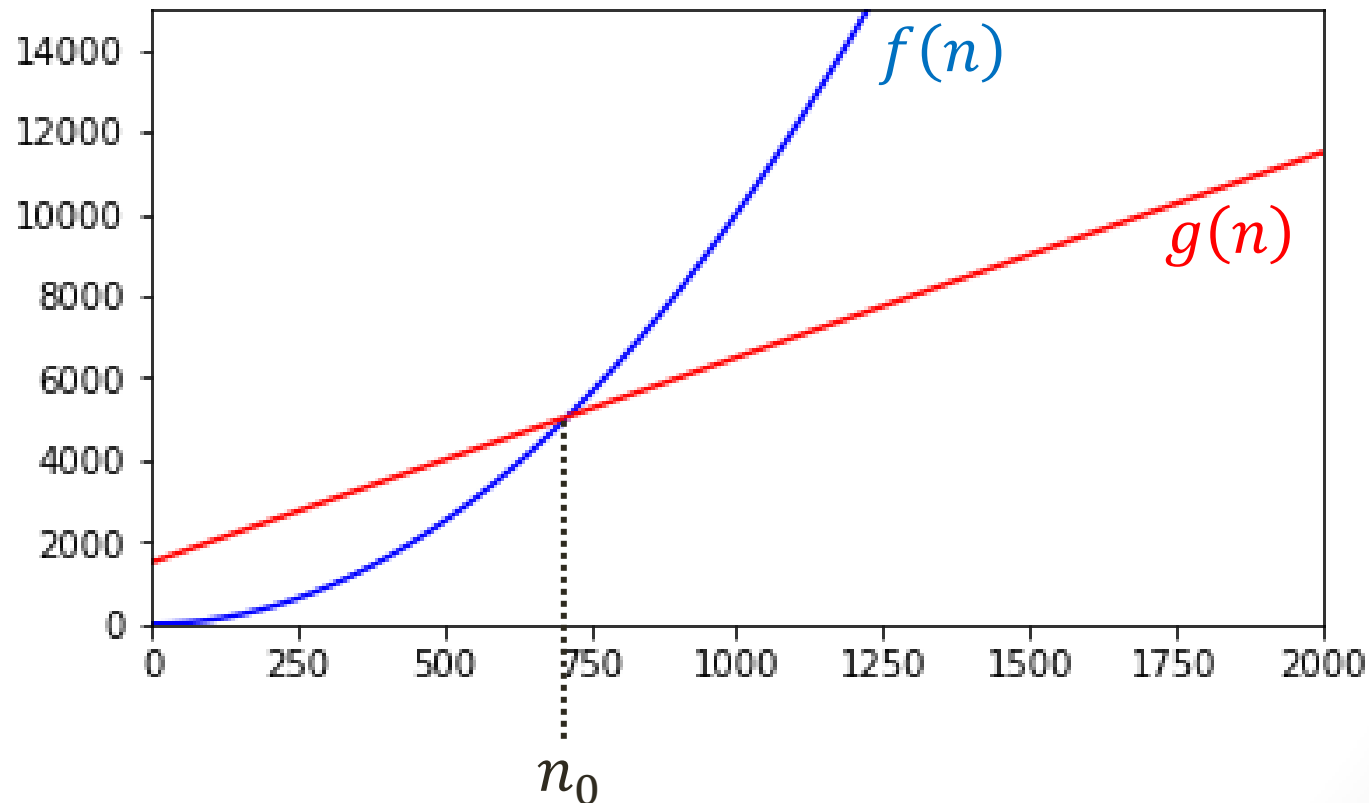


$$f(n) = 0,01n^2$$

$$g(n) = 5n + 1500$$

Complexidade de algoritmos

- Comportamento assintótico de funções



Complexidade de algoritmos

- Notações assintóticas
- Serão apresentadas, a seguir, um conjunto de notações que descrevem o comportamento assintótico de um algoritmo
- Estas notações descrevem um conjunto de funções de tempo de execução que são limitadas assintoticamente (por cima e/ou por baixo) por alguma função em particular
- Lembrando que a análise do comportamento assintótico é feita para valores de n suficientemente grandes
- Estas notações serão utilizadas ao longo desta disciplina

Complexidade de algoritmos

- Notação O

$$O(f(n)) = \{g(n) : \exists c \text{ e } n_0 : g(n) \leq cf(n), \forall n \geq n_0\}$$

onde c e n_0 são duas constantes positivas

- Uma função $g(n)$ é $O(f(n))$ se existem duas constantes positivas c e n_0 tais que $g(n) \leq cf(n)$, para todo $n \geq n_0$

Complexidade de algoritmos

- Notação O

$$O(f(n)) = \{g(n) : \exists c \text{ e } n_0 : g(n) \leq cf(n), \forall n \geq n_0\}$$

- $O(f(n))$ é um conjunto de todas as funções $g(n)$ que são assintoticamente limitadas **por cima** por $f(n)$.
- Dizemos que $g(n) = O(f(n))$ quando $g(n)$ é um membro de $O(f(n))$

Complexidade de algoritmos

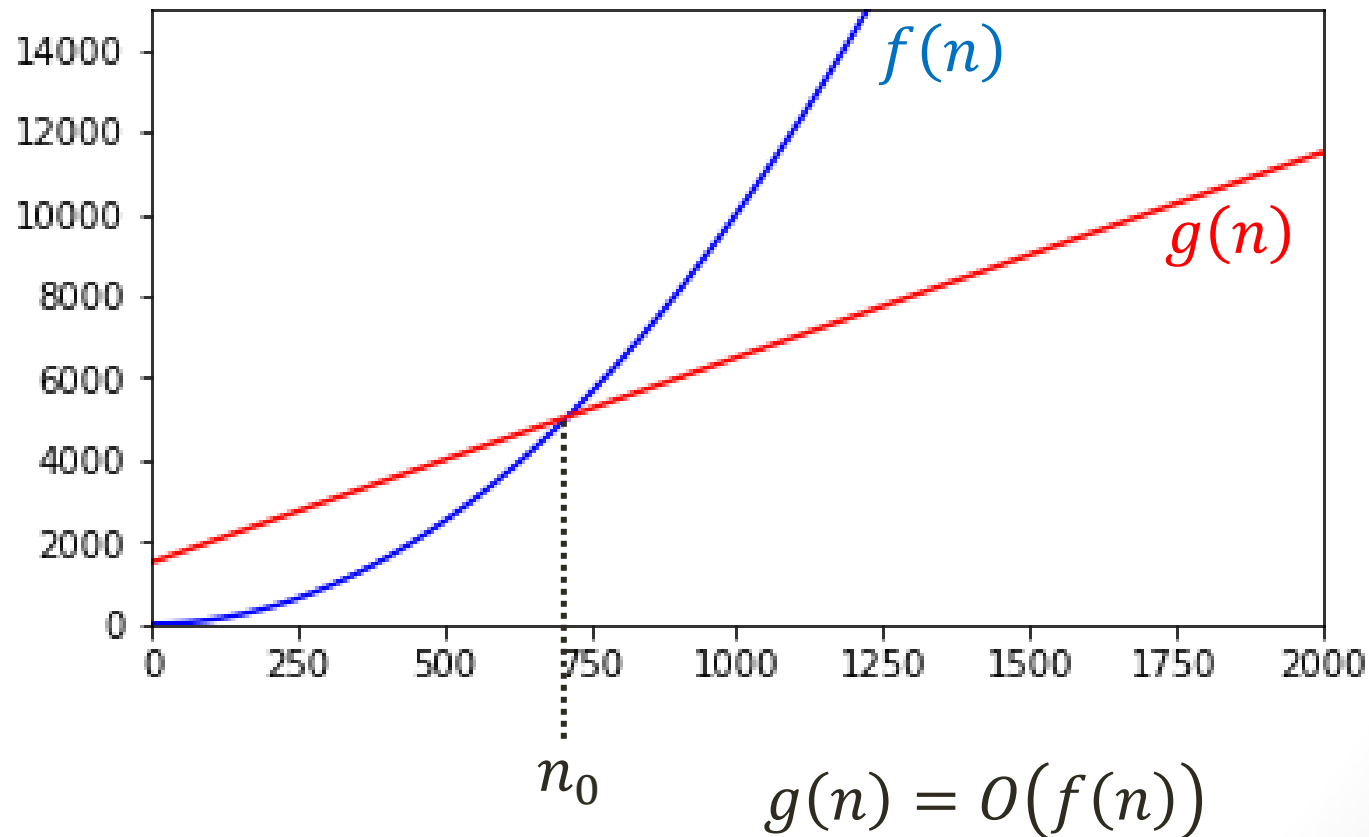
- Notação O

$$O(f(n)) = \{g(n) : \exists c \text{ e } n_0 : g(n) \leq cf(n), \forall n \geq n_0\}$$

- Aqui, a função $f(n)$ é um **limite assintótico superior** de todas as funções em $O(f(n))$
- Podemos empregar a notação O para limitar o **pior caso** de tempo de execução de um algoritmo

Complexidade de algoritmos

- Notação O



Complexidade de algoritmos

- Notação O
- $5n = O(n)$?

$c =$

$n_0 =$

SIM ☐

NÃO ☐

Complexidade de algoritmos

- Notação O

- $5n = O(n)$?

$$c = 5$$

$$n_0 = 0$$

SIM



NÃO



- $5n \leq cn, \forall n \geq n_0$?

$$c = 5$$

$$n_0 = 0$$

$$5n \leq 5n, \quad \forall n \geq 0$$

SIM

Complexidade de algoritmos

- Notação O
- $n + 10 = O(n)$?

$c =$

$n_0 =$

SIM ☐

NÃO ☐

Complexidade de algoritmos

- Notação O

- $n + 10 = O(n)$?

$$c = 2$$

$$n_0 = 10$$

SIM



NÃO



- $n + 10 \leq cn, \forall n \geq n_0$?

$$c = 2$$

$$n_0 = 10$$

$$n + 10 \leq 2n, \forall n \geq 10$$

SIM

Complexidade de algoritmos

- Notação O
- $3n + 2 = O(n)$?

$c =$

$n_0 =$

SIM ☐

NÃO ☐

Complexidade de algoritmos

- Notação O

- $3n + 2 = O(n)$?

$$c = 4$$

$$n_0 = 2$$

SIM



NÃO



- $3n + 2 \leq cn, \forall n \geq n_0$?

$$c = 4$$

$$n_0 = 2$$

$$3n + 2 \leq 4n, \forall n \geq 2$$

SIM

Complexidade de algoritmos

- Notação O
- $10n + 100 = O(n)$?

$c =$

$n_0 =$

SIM ☐

NÃO ☐

Complexidade de algoritmos

- Notação O

- $10n + 100 = O(n)$?

$$c = 11$$

$$n_0 = 100$$

SIM



NÃO



- $10n + 100 \leq cn, \forall n \geq n_0$?

$$c = 11$$

$$n_0 = 100$$

$$10n + 100 \leq 11n, \forall n \geq 100$$

SIM

Complexidade de algoritmos

- Notação O
- $n = O(n)$?

$c =$

$n_0 =$

SIM ☐

NÃO ☐

Complexidade de algoritmos

- Notação O

- $n = O(n)$?

$$c = 1$$

$$n_0 = 0$$

SIM



NÃO



- $n \leq cn, \forall n \geq n_0$?

$$c = 1$$

$$n_0 = 0$$

$$n \leq 1 \cdot n, \forall n \geq 0$$

SIM

Complexidade de algoritmos

- Notação O
- $5 = O(n)$?

$c =$

$n_0 =$

SIM ☐

NÃO ☐

Complexidade de algoritmos

- Notação O

- $5 = O(n)$?

$$c = 1$$

$$n_0 = 5$$

SIM



NÃO



- $5 \leq cn, \forall n \geq n_0$?

$$c = 1$$

$$n_0 = 5$$

$$5 \leq 1 \cdot n, \forall n \geq 5$$

SIM

Complexidade de algoritmos

- Notação O
- $n^2 = O(n)$?

$c =$

$n_0 =$

SIM ☐

NÃO ☐

Complexidade de algoritmos

- Notação O

- $n^2 = O(n)$?

$$c = \nexists$$

$$n_0 = \nexists$$

SIM



NÃO



- $n^2 \leq cn, \forall n \geq n_0$?

$$c = \nexists$$

$$n_0 = \nexists$$

NÃO

Complexidade de algoritmos

- Notação O
- $n = O(n^2)$?

$c =$

$n_0 =$

SIM ☐

NÃO ☐

Complexidade de algoritmos

- Notação O

- $n = O(n^2)$?

$$c = 1$$

$$n_0 = 0$$

SIM



NÃO



- $n \leq cn^2, \forall n \geq n_0$?

$$c = 1$$

$$n_0 = 0$$

$$n \leq 1 \cdot n^2, \forall n \geq 0$$

SIM

Complexidade de algoritmos

- Notação O
- $\log_2 n = O(n)$?

$c =$

$n_0 =$

SIM ☐

NÃO ☐

Complexidade de algoritmos

- Notação O

- $\log_2 n = O(n)$?

$$c = 1$$

$$n_0 = 1$$

SIM



NÃO



- $\log_2 n \leq cn, \forall n \geq n_0$?

$$c = 1$$

$$n_0 = 1$$

$$\log_2 n \leq 1 \cdot n, \forall n \geq 1$$

SIM

Complexidade de algoritmos

- Notação O
- $3 = O(1)$?

$c =$

$n_0 =$

SIM ☐

NÃO ☐

Complexidade de algoritmos

- Notação O

- $3 = O(1)$?

$$c = 3$$

$$n_0 = 0$$

SIM



NÃO



- $3 \leq c \cdot 1, \forall n \geq n_0$?

$$c = 3$$

$$n_0 = 0$$

$$3 \leq 3 \cdot 1, \forall n \geq 0$$

SIM

Complexidade de algoritmos

- Notação O
- $n = O(1)$?

$c =$

$n_0 =$

SIM ☐

NÃO ☐

Complexidade de algoritmos

- Notação O

- $n = O(1)$?

$$c = \nexists$$

$$n_0 = \nexists$$

SIM

☐

NÃO

☐

- $n \leq c \cdot 1, \forall n \geq n_0$?

$$c = \nexists$$

$$n_0 = \nexists$$

NÃO

Complexidade de algoritmos

- Notação O
- $n^2 = O(n^2)$?

$c =$

$n_0 =$

SIM ☐

NÃO ☐

Complexidade de algoritmos

- Notação O

- $n^2 = O(n^2)$?

$$c = 1$$

$$n_0 = 0$$

SIM



NÃO



- $n^2 \leq cn^2, \forall n \geq n_0$?

$$c = 1$$

$$n_0 = 0$$

$$n^2 \leq 1 \cdot n^2, \forall n \geq 0$$

SIM

Complexidade de algoritmos

- Notação O
- $n^2 = O(n^3)$?

$c =$

$n_0 =$

SIM ☐

NÃO ☐

Complexidade de algoritmos

- Notação O

- $n^2 = O(n^3)$?

$$c = 1$$

$$n_0 = 0$$

SIM



NÃO



- $n^2 \leq cn^3, \forall n \geq n_0$?

$$c = 1$$

$$n_0 = 0$$

$$n^2 \leq 1 \cdot n^3, \forall n \geq 0$$

SIM

Complexidade de algoritmos

- Notação O
- $\log_3 n = O(\log_2 n)$?

$c =$

$n_0 =$

SIM ☐

NÃO ☐

Complexidade de algoritmos

- Notação O

- $\log_3 n = O(\log_2 n)$?

$$c = \frac{1}{\log_2 3}$$

$$n_0 = 1$$

SIM



NÃO



- $\log_3 n \leq c \log_2 n, \forall n \geq n_0$?

- $\frac{\log_2 n}{\log_2 3} \leq c \log_2 n, \forall n \geq n_0$?

$$c = \frac{1}{\log_2 3}$$

$$n_0 = 1$$

$$\log_3 n \leq \frac{1}{\log_2 3} \log_2 n, \forall n \geq 1$$

SIM

Complexidade de algoritmos

- Notação O

Classe de problema	Complexidade
$T(n) = O(1)$	constante
$T(n) = O(\log_2 n)$	logarítmica
$T(n) = O(n)$	linear
$T(n) = O(n \log_2 n)$	linearítmica
$T(n) = O(n^2)$	quadrática
$T(n) = O(n^3)$	cúbica
$T(n) = O(2^n)$	exponencial
$T(n) = O(n!)$	fatorial

Complexidade de algoritmos

- Notação O

Classe de problema	$n = 128$
$T(n) = O(1)$	c instruções
$T(n) = O(\log_2 n)$	$\propto 7$ instruções
$T(n) = O(n)$	$\propto 128$ instruções
$T(n) = O(n \log_2 n)$	$\propto 896$ instruções
$T(n) = O(n^2)$	$\propto 16.384$ instruções
$T(n) = O(n^3)$	$\propto 2.097.152$ instruções
$T(n) = O(2^n)$	$\propto 3,4 \cdot 10^{38}$ instruções
$T(n) = O(n!)$	$\propto 3,85 \cdot 10^{215}$ instruções

Complexidade de algoritmos

- Notação Ω

$$\Omega(f(n)) = \{g(n) : \exists c \text{ e } n_0 : g(n) \geq cf(n), \forall n \geq n_0\}$$

onde c e n_0 são duas constantes positivas

- Uma função $g(n)$ é $\Omega(f(n))$ se existem duas constantes positivas c e n_0 tais que $g(n) \geq cf(n)$, para todo $n \geq n_0$

Complexidade de algoritmos

- Notação Ω

$$\Omega(f(n)) = \{g(n) : \exists c \text{ e } n_0 : g(n) \geq cf(n), \forall n \geq n_0\}$$

- $\Omega(f(n))$ é um conjunto de todas as funções $g(n)$ que são assintoticamente limitadas **por baixo** por $f(n)$.
- Dizemos que $g(n) = \Omega(f(n))$ quando $g(n)$ é um membro de $\Omega(f(n))$

Complexidade de algoritmos

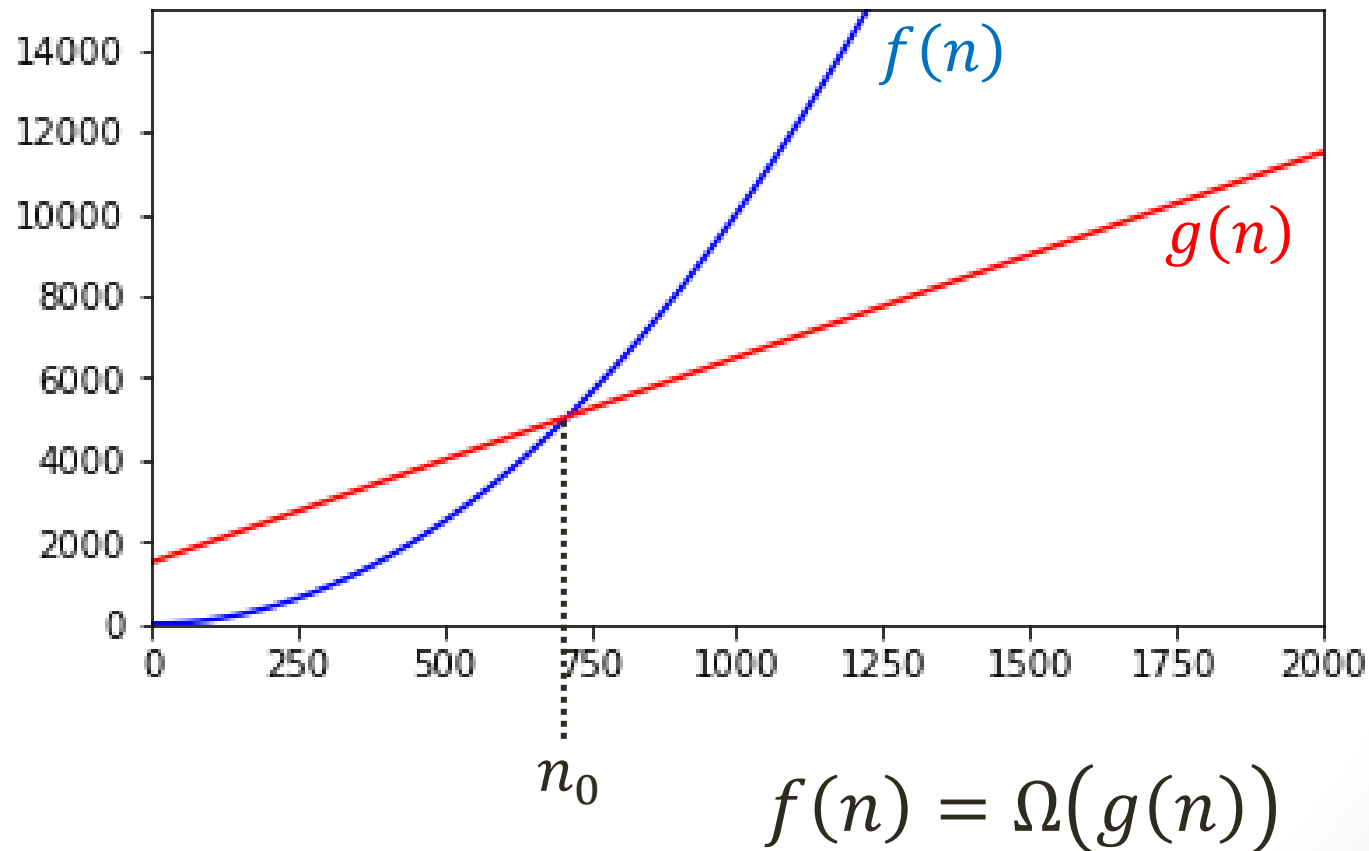
- Notação Ω

$$\Omega(f(n)) = \{g(n) : \exists c \text{ e } n_0 : g(n) \geq cf(n), \forall n \geq n_0\}$$

- Aqui, a função $f(n)$ é um **limite assintótico inferior** de todas as funções em $\Omega(f(n))$
- Podemos empregar a notação Ω para limitar o **melhor caso** de tempo de execução de um algoritmo

Complexidade de algoritmos

- Notação Ω



Complexidade de algoritmos

- Notação Ω
- $n = \Omega(n^2)$?

$c =$

$n_0 =$

SIM ☐

NÃO ☐

Complexidade de algoritmos

- Notação Ω

- $n = \Omega(n^2)$?

$$c = \nexists$$

$$n_0 = \nexists$$

SIM

☐

NÃO

☐

- $n \geq cn^2, \forall n \geq n_0$?

$$c = \nexists$$

$$n_0 = \nexists$$

NÃO

Complexidade de algoritmos

- Notação Ω
- $n^2 = \Omega(n)$?

$c =$

$n_0 =$

SIM ☐

NÃO ☐

Complexidade de algoritmos

- Notação Ω

- $n^2 = \Omega(n)$?

$$c = 1$$

$$n_0 = 0$$

SIM



NÃO



- $n^2 \geq cn, \forall n \geq n_0$?

$$c = 1$$

$$n_0 = 0$$

$$n^2 \geq 1 \cdot n, \forall n \geq 0$$

SIM

Complexidade de algoritmos

- Notação Ω
- $n = \Omega(n)$?

$c =$

$n_0 =$

SIM ☐

NÃO ☐

Complexidade de algoritmos

- Notação Ω

- $n = \Omega(n)$?

$$c = 1$$

$$n_0 = 0$$

SIM



NÃO



- $n \geq cn, \forall n \geq n_0$?

$$c = 1$$

$$n_0 = 0$$

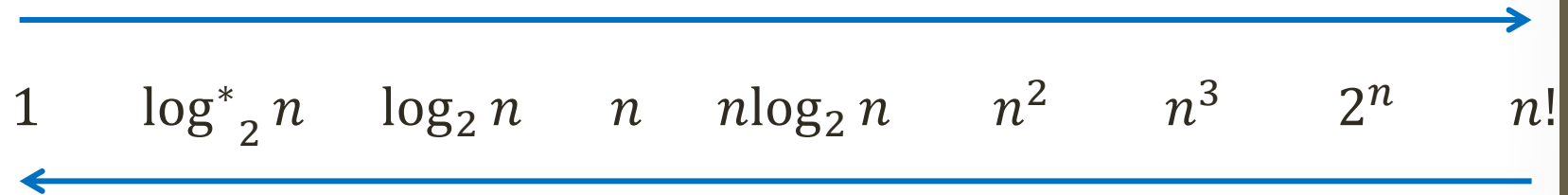
$$n \geq 1 \cdot n, \forall n \geq 0$$

SIM

Complexidade de algoritmos

- Hierarquia de algumas funções

$$O : f(n) = O(g(n))$$



$$\Omega : g(n) = \Omega(f(n))$$

Complexidade de algoritmos

- Notação Θ

$$\Theta(f(n)) = \{g(n) : \exists c_1, c_2 \text{ e } n_0 : c_1 f(n) \leq g(n) \leq c_2 f(n), \forall n \geq n_0\}$$

onde c_1 , c_2 e n_0 são três constantes positivas

- Uma função $g(n)$ é $\Theta(f(n))$ se existem três constantes positivas c_1 , c_2 e n_0 tais que $c_1 f(n) \leq g(n) \leq c_2 f(n)$, para todo $n \geq n_0$

Complexidade de algoritmos

- Notação Θ

$$\Theta(f(n)) = \{g(n) : \exists c_1, c_2 \text{ e } n_0 : c_1 f(n) \leq g(n) \leq c_2 f(n), \forall n \geq n_0\}$$

$$\Theta(f(n)) = O(f(n)) \cap \Omega(f(n))$$

Complexidade de algoritmos

- Notação Θ

$$\Theta(f(n)) = \{g(n) : \exists c_1, c_2 \text{ e } n_0 : c_1 f(n) \leq g(n) \leq c_2 f(n), \forall n \geq n_0\}$$

- $\Theta(f(n))$ é um conjunto de todas as funções $g(n)$ que são assintoticamente limitadas **por baixo** e **por cima** ao mesmo tempo (dadas duas constantes c_1 e c_2).
- Dizemos que $g(n) = \Theta(f(n))$ quando $g(n)$ é um membro de $\Theta(f(n))$

Complexidade de algoritmos

- Notação Θ

$$\Theta(f(n)) = \{g(n) : \exists c_1, c_2 \text{ e } n_0 : c_1 f(n) \leq g(n) \leq c_2 f(n), \forall n \geq n_0\}$$

- Aqui, a função $f(n)$ é um **limite assintótico firme** de todas as funções em $\Theta(f(n))$

Complexidade de algoritmos

- Uma forma de simplificação em duas etapas:

$$T(n) = 25n^2 + 108n \log_2 n + 1539n + 14774 \log_2 n + 98032$$

$$T(n) = 25n^2 + \cancel{108n \log_2 n} + \cancel{1539n} + \cancel{14774 \log_2 n} + \cancel{98032}$$

$$T(n) = 25n^2 + 108n \log_2 n + 1539n + 14774 \log_2 n + 98032$$

$$T(n) = \cancel{25n^2} + 108n \log_2 n + 1539n + 14774 \log_2 n + 98032$$

$$T(n) = 25n^2 + 108n \log_2 n + 1539n + 14774 \log_2 n + 98032$$

$$T(n) = O(n^2)$$

Complexidade de algoritmos

- Recursão
- Quando um objeto é definido em termos de si mesmo, dizemos que houve uma recursão. Dizemos, também, que tal objeto é recursivo
- A recursão é dividida em dois casos:
 - Caso geral (ou recursivo, propriamente dito), onde o objeto é definido em termos de si mesmo
 - Caso base, onde o objeto pode ser suficientemente definido de forma não-recursiva

Complexidade de algoritmos

- Recursão
- A recursão é dividida em dois casos:
 - Caso geral
 - Caso base
- Exemplo : Árvore
 - Caso geral: uma árvore é uma estrutura que consiste de um nó (chamado raiz) e neste nó estão penduradas outras árvores
 - Caso base: uma árvore é uma estrutura vazia

Complexidade de algoritmos

- Recursão
- A recursão é dividida em dois casos:
 - Caso geral
 - Caso base
- Exemplo : Lista (listas também podem ser definidas recursivamente)
 - Caso geral: uma lista é uma estrutura que consiste de um elemento inicial (chamado cabeça) sucedido por uma lista (chamada cauda)
 - Caso base: uma lista é uma estrutura vazia

Complexidade de algoritmos

- Recursão
- Um algoritmo recursivo também é dividido em dois casos:
 - Caso geral: o algoritmo chama a si mesmo, resolvendo problemas com tamanhos menores
 - Caso base: o tamanho do problema é pequeno o suficiente para requerer uma solução não-recursiva

Complexidade de algoritmos

```
(Alg. principal)  
:  
a ← Fatorial(4);  
:
```

Complexidade de algoritmos

(Alg. principal)

⋮

a ← **Fatorial(4);**

⋮

(n = 4)



Fatorial(n);

se n > 1

então | Fatorial ← n * Fatorial(n - 1);

senão | Fatorial ← 1;

Complexidade de algoritmos

(Alg. principal)
:
a ← **Fatorial(4)**;
:

(n = 4)



```
Fatorial(n);  
    se n > 1  
    então | Fatorial ← n * Fatorial(n - 1);  
    senão  | Fatorial ← 1;
```

```
Fatorial(n);  
    se n > 1  
    então | Fatorial ← n * Fatorial(n - 1);  
    senão  | Fatorial ← 1;
```



(n = 3)

Complexidade de algoritmos

(Alg. principal)
:
a ← Fatorial(4);
:

(n = 4)



```
Fatorial(n);  
    se n > 1  
    então | Fatorial ← n * Fatorial(n - 1);  
    senão | Fatorial ← 1;
```

(n = 3)



```
Fatorial(n);  
    se n > 1  
    então | Fatorial ← n * Fatorial(n - 1);  
    senão | Fatorial ← 1;
```

(n = 2)



```
Fatorial(n);  
    se n > 1  
    então | Fatorial ← n * Fatorial(n - 1);  
    senão | Fatorial ← 1;
```

Complexidade de algoritmos

(Alg. principal)
:
a ← Fatorial(4);
:

(n = 4)



```
Fatorial(n);  
    se n > 1  
    então | Fatorial ← n * Fatorial(n - 1);  
    senão | Fatorial ← 1;
```

(n = 3)



```
Fatorial(n);  
    se n > 1  
    então | Fatorial ← n * Fatorial(n - 1);  
    senão | Fatorial ← 1;
```

(n = 2)



```
Fatorial(n);  
    se n > 1  
    então | Fatorial ← n * Fatorial(n - 1);  
    senão | Fatorial ← 1;
```

(n = 1)



```
Fatorial(n);  
    se n > 1  
    então | Fatorial ← n * Fatorial(n - 1);  
    senão | Fatorial ← 1;
```

Complexidade de algoritmos

(Alg. principal)
:
a ← Fatorial(4);
:

(n = 4)



```
Fatorial(n);  
    se n > 1  
    então | Fatorial ← n * Fatorial(n - 1);  
    senão | Fatorial ← 1;
```

(n = 3)



```
Fatorial(n);  
    se n > 1  
    então | Fatorial ← n * Fatorial(n - 1);  
    senão | Fatorial ← 1;
```

(n = 2)



```
Fatorial(n);  
    se n > 1  
    então | Fatorial ← n * Fatorial(n - 1);  
    senão | Fatorial ← 1;
```

(retorna 1)



```
Fatorial(n);  
    se n > 1  
    então | Fatorial ← n * Fatorial(n - 1);  
    senão | Fatorial ← 1;
```

Complexidade de algoritmos

(Alg. principal)
:
a ← Fatorial(4);
:

(n = 4)



```
Fatorial(n);  
    se n > 1  
    então | Fatorial ← n * Fatorial(n - 1);  
    senão | Fatorial ← 1;
```

(n = 3)



```
Fatorial(n);  
    se n > 1  
    então | Fatorial ← n * Fatorial(n - 1);  
    senão | Fatorial ← 1;
```

(retorna 2)



```
Fatorial(n);  
    se n > 1  
    então | Fatorial ← n * Fatorial(n - 1);  
    senão | Fatorial ← 1;
```

Complexidade de algoritmos

(Alg. principal)
:
a ← Fatorial(4);
:

(n = 4)



Fatorial(n);
 se n > 1
 então | **Fatorial** ← n * Fatorial(n - 1);
 senão | Fatorial ← 1;

Fatorial(n);
 se n > 1
 então | Fatorial ← n * Fatorial(n - 1);
 senão | Fatorial ← 1;



(retorna 6)

Complexidade de algoritmos

(Alg. principal)

⋮

a ← Fatorial(4);

⋮

(retorna **24**)



Fatorial(n);

se $n > 1$

então | Fatorial ← $n * \text{Fatorial}(n - 1)$;

senão | Fatorial ← 1;

Complexidade de algoritmos

```
(Alg. principal)  
:  
a ← Fatorial(4);  
:
```

Complexidade de algoritmos

	<u>nº execuções</u>
Fatorial(n);	
<u>se</u> n > 1	
<u>então</u> Fatorial ← n * Fatorial(n - 1);	
<u>senão</u> Fatorial ← 1;	

$$T(n) =$$

Complexidade de algoritmos

		<u>nº execuções</u>
Fatorial(n);		
<u>se</u> $n > 1$		1
<u>então</u>	Fatorial $\leftarrow n * \text{Fatorial}(n - 1);$	0 <u>ou</u> $T(n - 1)$
<u>senão</u>	Fatorial $\leftarrow 1;$	1 <u>ou</u> 0

$$T(n) = \begin{cases} T(n - 1) + 1, & \text{se } n > 1 \\ 2, & \text{caso contrário} \end{cases}$$

Complexidade de algoritmos

Exemplo(n);		<u>nº execuções</u>
<u>se</u> $n > 1$		
<u>então</u>	$a \leftarrow \text{Exemplo}(n/2);$	
	$b \leftarrow \text{Exemplo}(n/2);$	
	$\text{Exemplo} \leftarrow a + b;$	
<u>senão</u>	$\text{Exemplo} \leftarrow 1;$	

$$T(n) =$$

Complexidade de algoritmos

Exemplo(n);		<u>nº execuções</u>
<u>se</u> $n > 1$		1
<u>então</u>	$a \leftarrow \text{Exemplo}(n/2);$	0 <u>ou</u> $T(n/2)$
	$b \leftarrow \text{Exemplo}(n/2);$	0 <u>ou</u> $T(n/2)$
	$\text{Exemplo} \leftarrow a + b;$	0 <u>ou</u> 1
<u>senão</u>	$\text{Exemplo} \leftarrow 1;$	1 <u>ou</u> 0

$$T(n) = \begin{cases} 2T(n/2) + 2, & \text{se } n > 1 \\ 2, & \text{caso contrário} \end{cases}$$

Complexidade de algoritmos

- Os casos base das funções de tempo de execução recursivas geralmente ocorrem para pequenos valores de n
- Nesta disciplina, os casos base serão omitidos nas funções $T(n)$

$$T(n) = \begin{cases} T(n - 1) + 1, & \text{se } n > 1 \\ 2, & \text{caso contrário} \end{cases}$$



$$T(n) = T(n - 1) + 1$$

Complexidade de algoritmos

- Os casos base das funções de tempo de execução recursivas geralmente ocorrem para pequenos valores de n
- Nesta disciplina, os casos base serão omitidos nas funções $T(n)$

$$T(n) = \begin{cases} 2T(n/2) + 2, & \text{se } n > 1 \\ 2, & \text{caso contrário} \end{cases}$$



$$T(n) = 2T(n/2) + 2$$

Complexidade de algoritmos

- Recursão
- Limites assintóticos de algoritmos recursivos
- Três métodos muito utilizados para o cálculo de limites assintóticos recursivos:
 - Substituição
 - Árvore de recursão
 - Teorema Mestre
- Estudaremos neste curso o método da árvore de recursão

Complexidade de algoritmos

- Árvore de recursão
- Na árvore de recursão, um nó é valorado pelo custo $T(k)$, $k \leq n$, para resolver um subproblema de tamanho k recursivamente
- A raiz da árvore é valorada com $T(n)$
- As folhas da árvore terão custo $T(1)$. Assumiremos aqui que $T(1) = \Theta(1)$
- O custo total da árvore é dado pela soma dos custos de todos os nós

Complexidade de algoritmos

- Árvore de recursão
- Etapas:
 1. Construir a árvore de recursão, nível a nível
 2. Somar os custos de todos os nós da árvore

Complexidade de algoritmos

$$T(n) = T(n - 1) + 1 \quad \longrightarrow \quad \textcircled{T(n)}$$

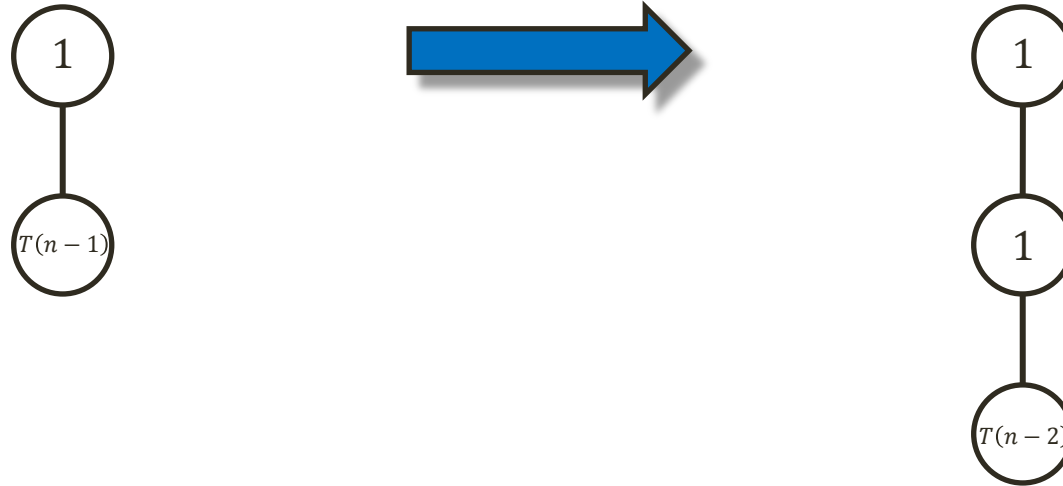
Complexidade de algoritmos

$T(n)$



$$T(n) = T(n - 1) + 1$$

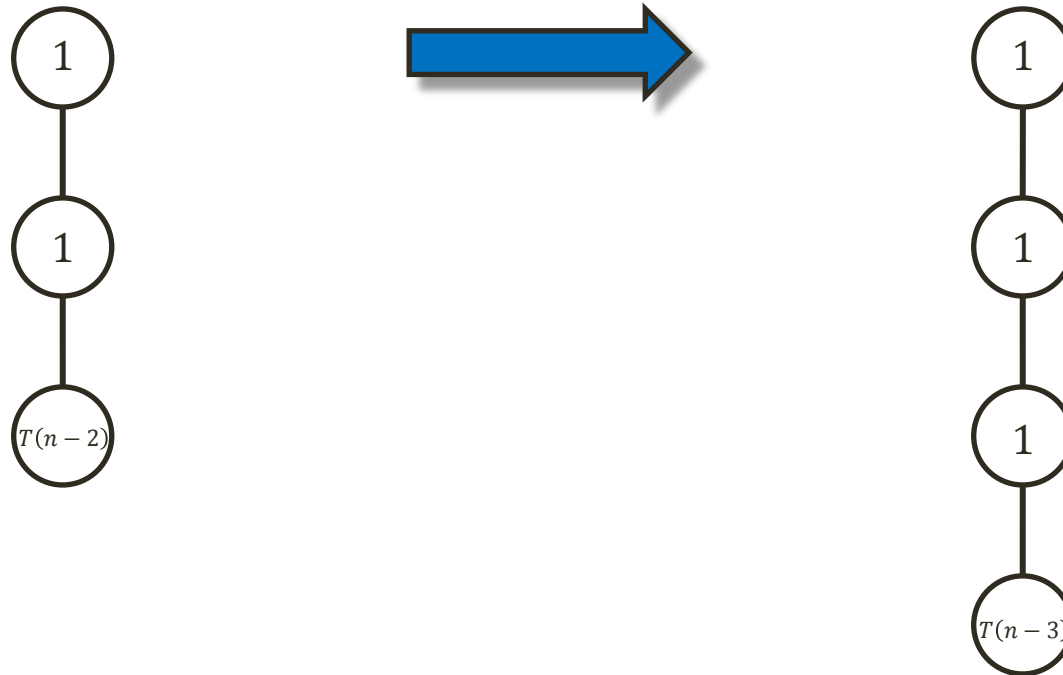
Complexidade de algoritmos



$$T(n) = T(n - 1) + 1$$

$$T(n - 1) = T(n - 2) + 1$$

Complexidade de algoritmos



$$T(n) = T(n - 1) + 1$$

$$T(n - 2) = T(n - 3) + 1$$

Complexidade de algoritmos



$$T(n) = T(n - 1) + 1$$

$$T(2) = T(1) + 1$$

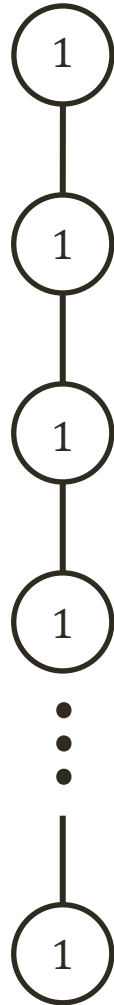
Complexidade de algoritmos



$$T(n) = T(n - 1) + 1$$

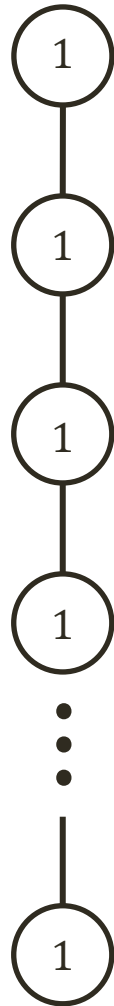
$$T(1) = 1$$

Complexidade de algoritmos



$$T(n) = T(n - 1) + 1$$

Complexidade de algoritmos



$$T(n) = T(n - 1) + 1$$

(A altura desta árvore é n)

$$T(n) = 1 \cdot (\text{altura da árvore})$$

$$T(n) = 1 \cdot n$$

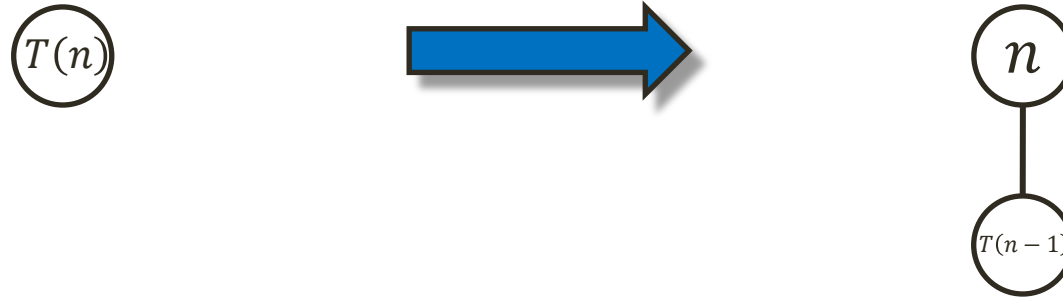
$$T(n) = n$$

$$T(n) = O(n)$$

Complexidade de algoritmos

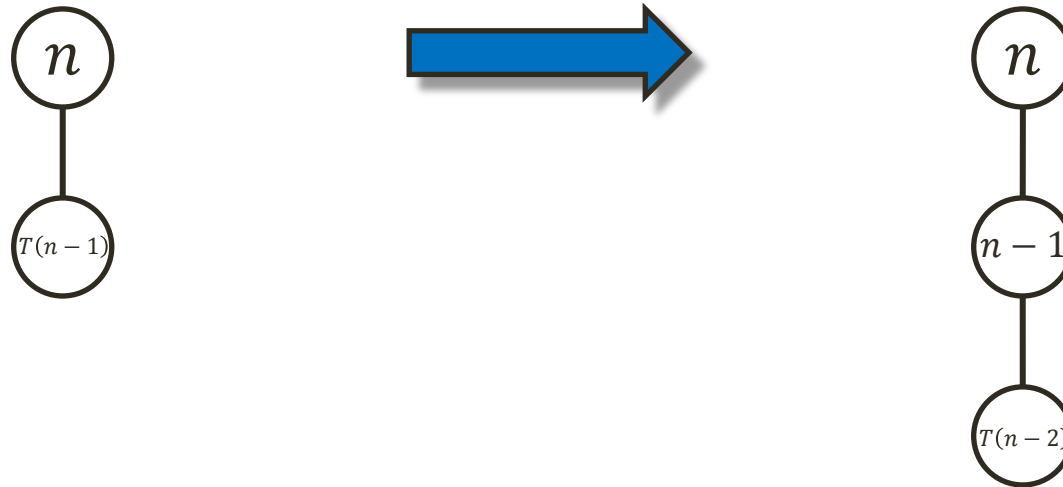
$$T(n) = T(n - 1) + n \quad \longrightarrow \quad \textcircled{T(n)}$$

Complexidade de algoritmos



$$T(n) = T(n - 1) + n$$

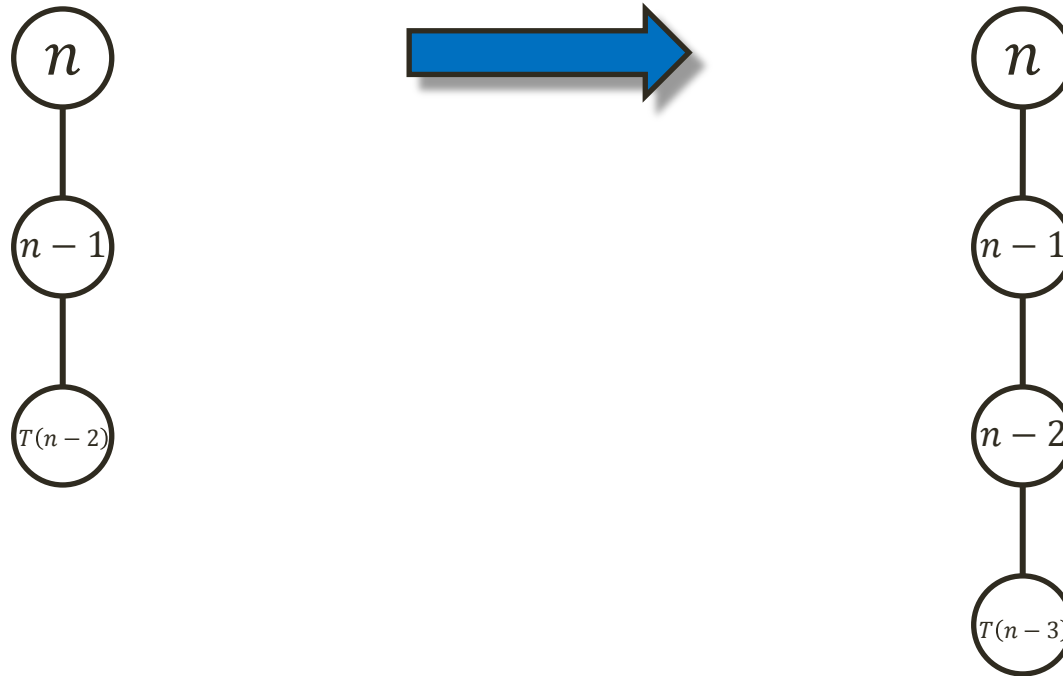
Complexidade de algoritmos



$$T(n) = T(n-1) + n$$

$$T(n-1) = T(n-2) + n-1$$

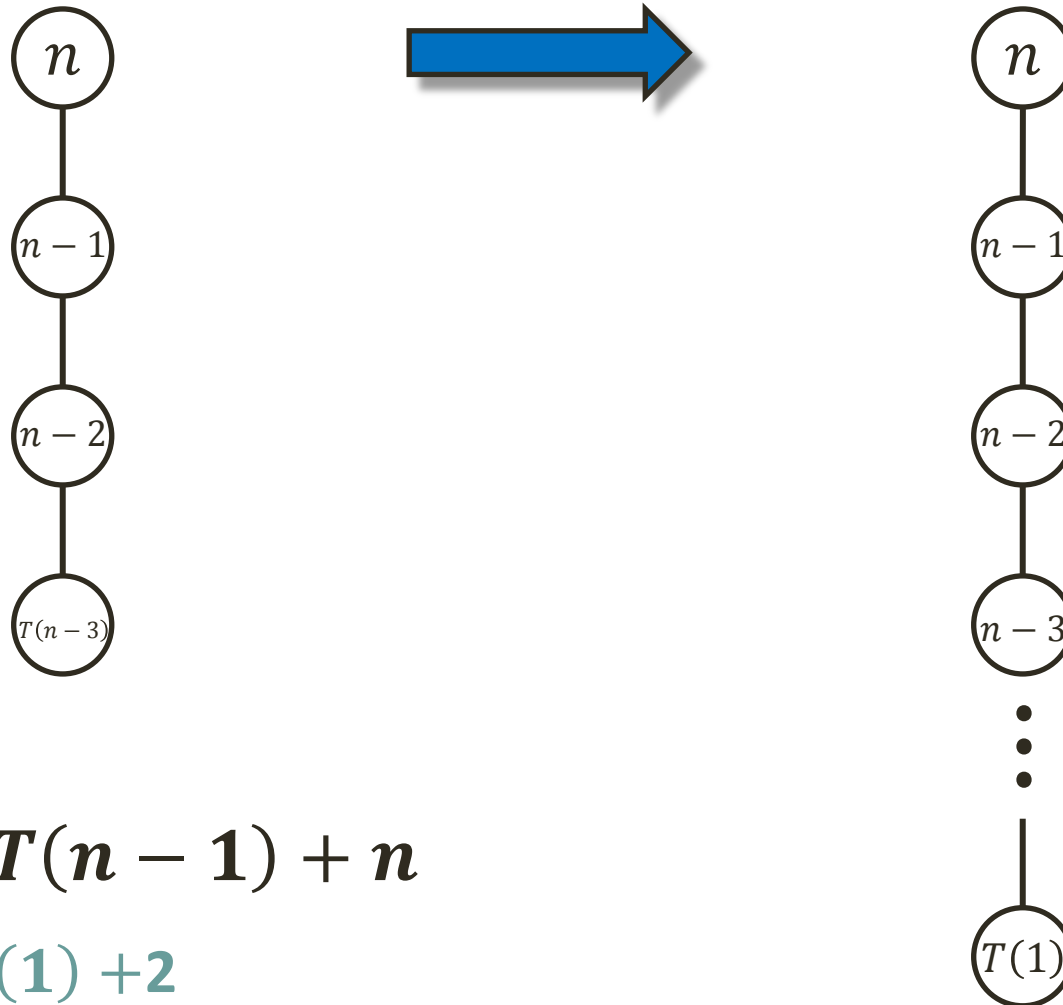
Complexidade de algoritmos



$$T(n) = T(n-1) + n$$

$$T(n-2) = T(n-3) + n-2$$

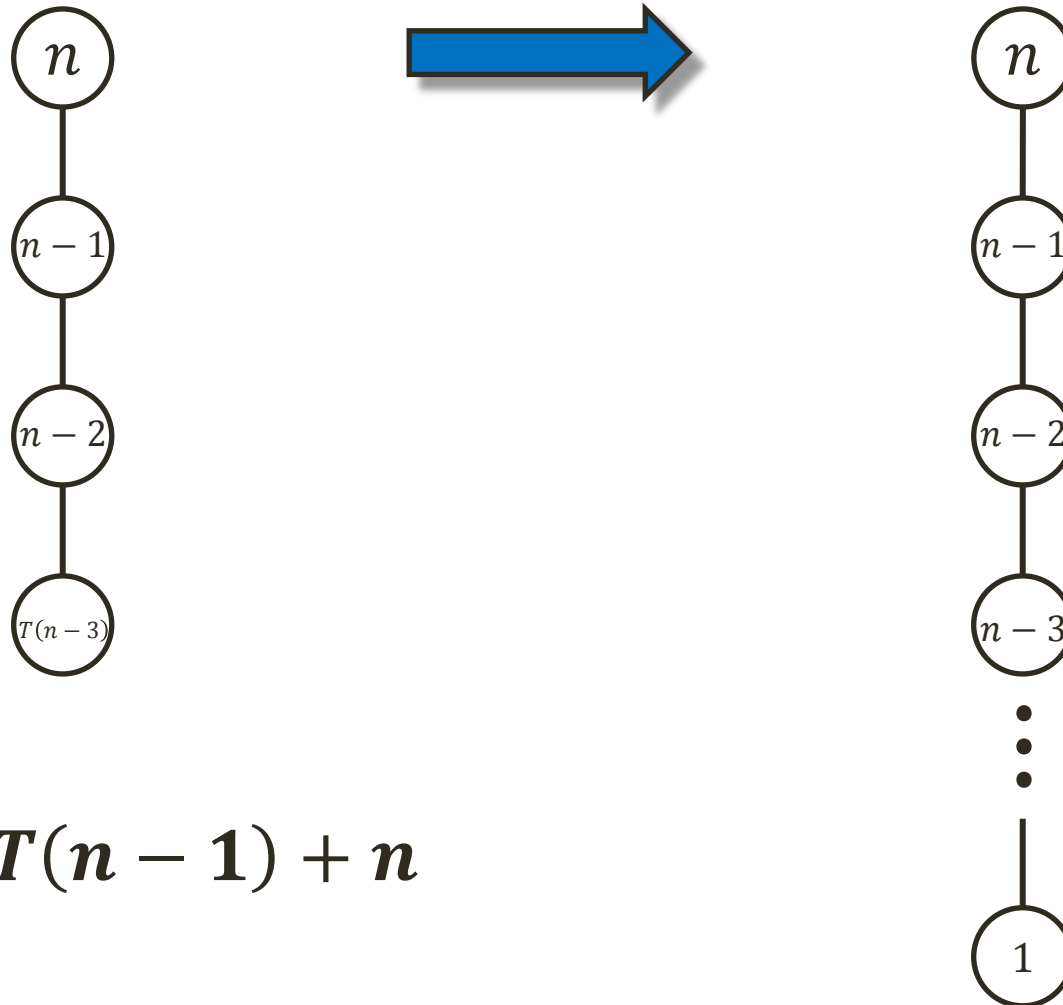
Complexidade de algoritmos



$$T(n) = T(n-1) + n$$

$$T(2) = T(1) + 2$$

Complexidade de algoritmos



$$T(n) = T(n-1) + n$$

$$T(1) = 1$$

Complexidade de algoritmos

n

$n - 1$

$n - 2$

$n - 3$

$\vdots$

1

$$T(n) = T(n - 1) + n$$

Complexidade de algoritmos



$$T(n) = T(n - 1) + n$$

(A altura desta árvore é n)

$$T(n) = 1 + 2 + 3 + \dots + n$$

$$S_n = \frac{(a_1 + a_n) \cdot n}{2}$$

(Soma dos termos de uma P.A.)

$$T(n) = \frac{(1 + n) \cdot n}{2}$$

$$T(n) = \frac{n^2}{2} + \frac{n}{2}$$

$$T(n) = O(n^2)$$

Complexidade de algoritmos

$$T(n) = T(n/2) + 1 \quad \longrightarrow \quad \textcircled{T(n)}$$

Complexidade de algoritmos

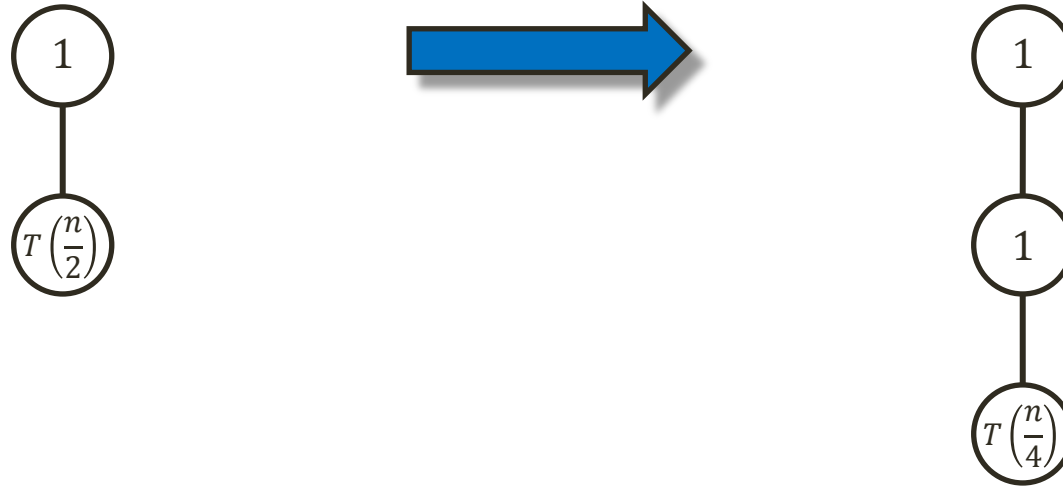
$T(n)$



1
 $T\left(\frac{n}{2}\right)$

$$T(n) = T(n/2) + 1$$

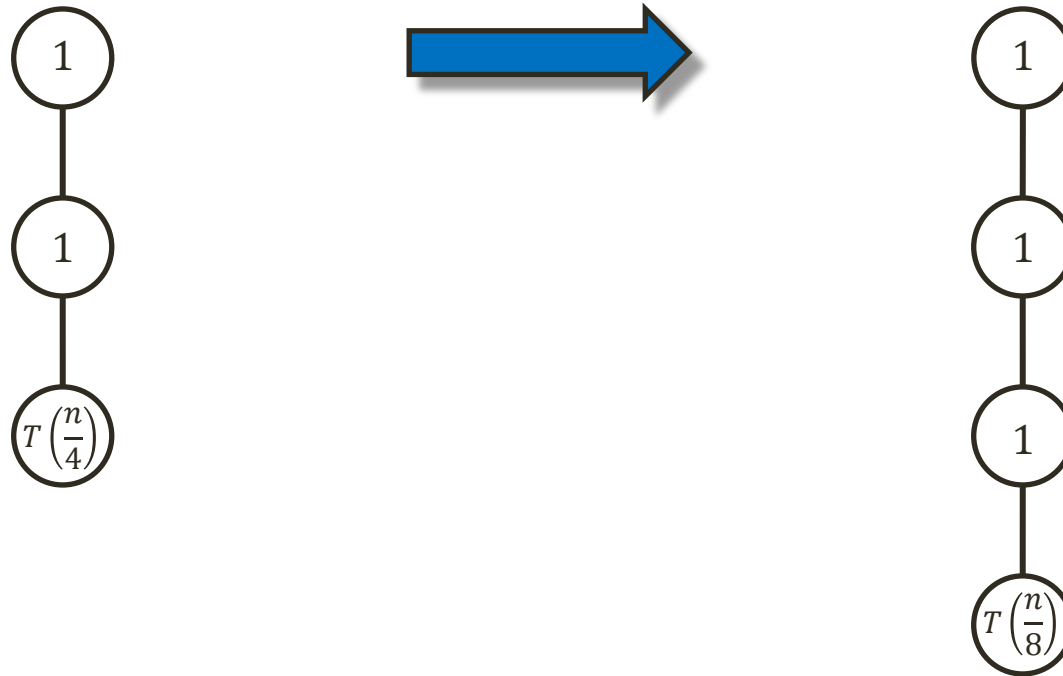
Complexidade de algoritmos



$$T(n) = T(n/2) + 1$$

$$T(n/2) = T(n/4) + 1$$

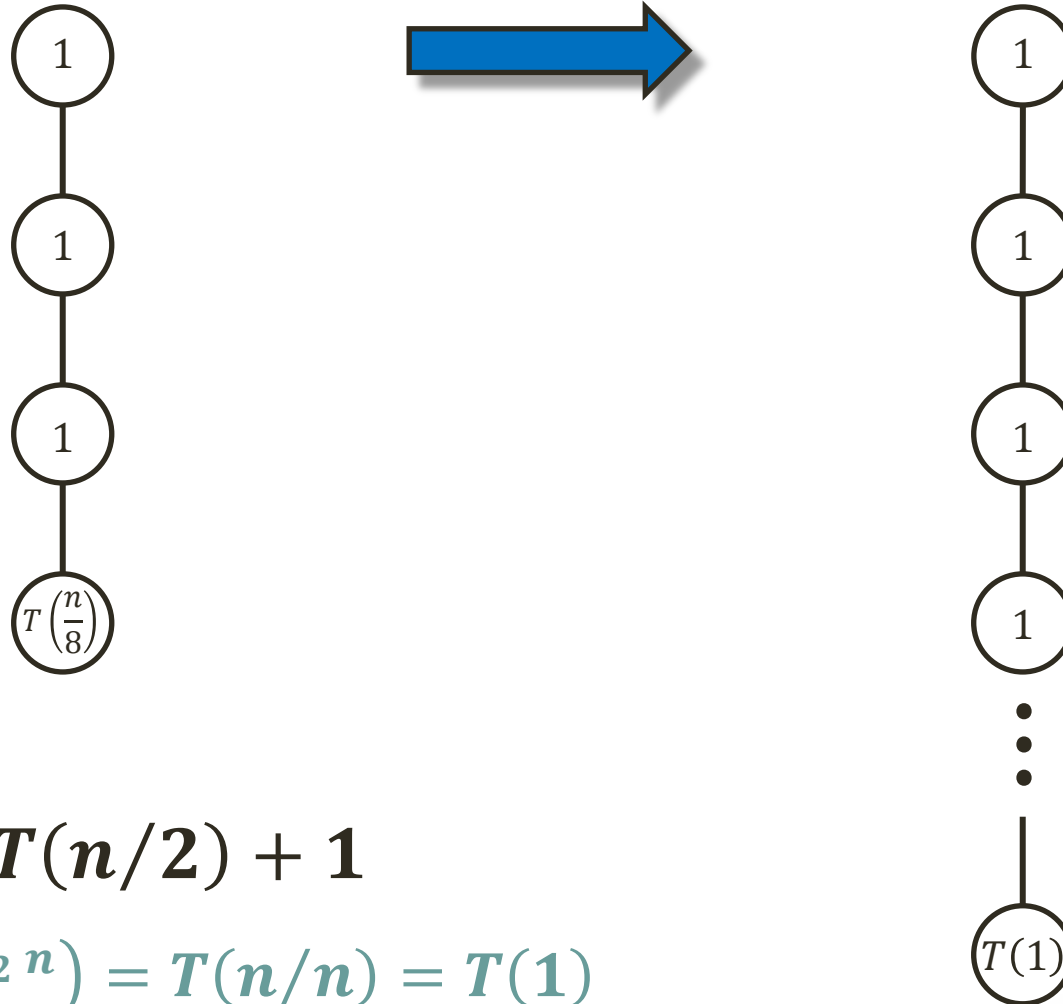
Complexidade de algoritmos



$$T(n) = T(n/2) + 1$$

$$T(n/4) = T(n/8) + 1$$

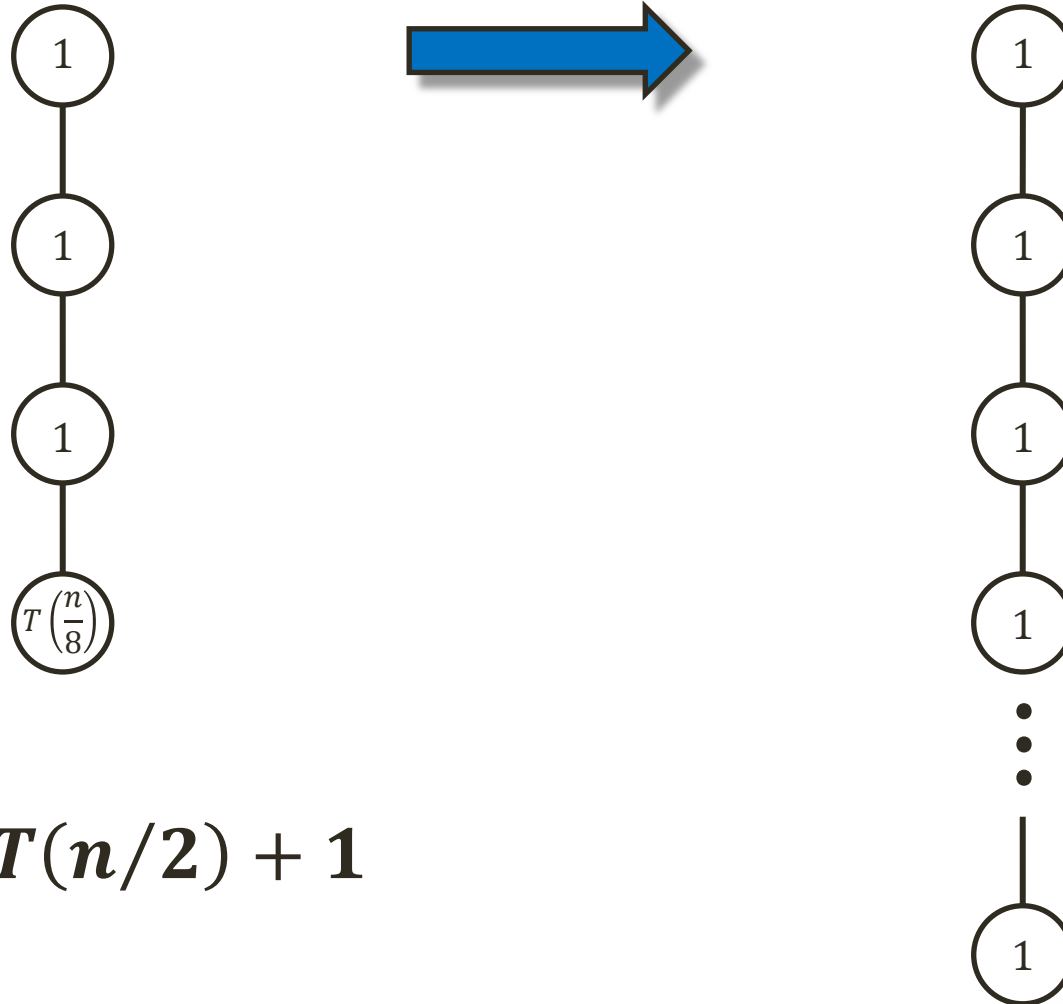
Complexidade de algoritmos



$$T(n) = T(n/2) + 1$$

$$T(n/2^{\log_2 n}) = T(n/n) = T(1)$$

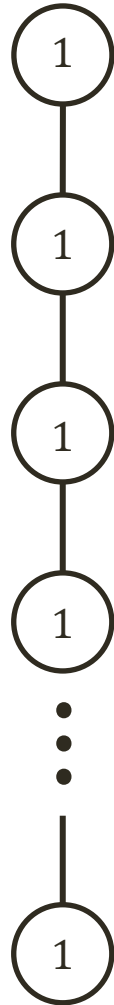
Complexidade de algoritmos



$$T(n) = T(n/2) + 1$$

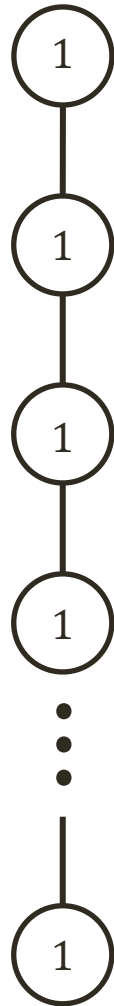
$$T(1) = 1$$

Complexidade de algoritmos



$$T(n) = T(n/2) + 1$$

Complexidade de algoritmos



$$T(n) = T(n/2) + 1$$

(A altura desta árvore é $\log_2 n + 1$)

$$T(n) = 1 \cdot (\text{altura da árvore})$$

$$T(n) = 1 \cdot (\log_2 n + 1)$$

$$T(n) = \log_2 n + 1$$

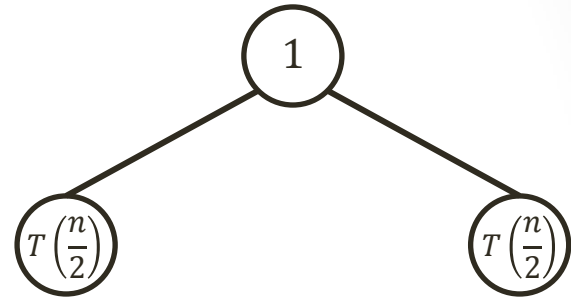
$$T(n) = O(\log_2 n)$$

Complexidade de algoritmos

$$T(n) = 2T(n/2) + 1 \quad \longrightarrow \quad \textcircled{T(n)}$$

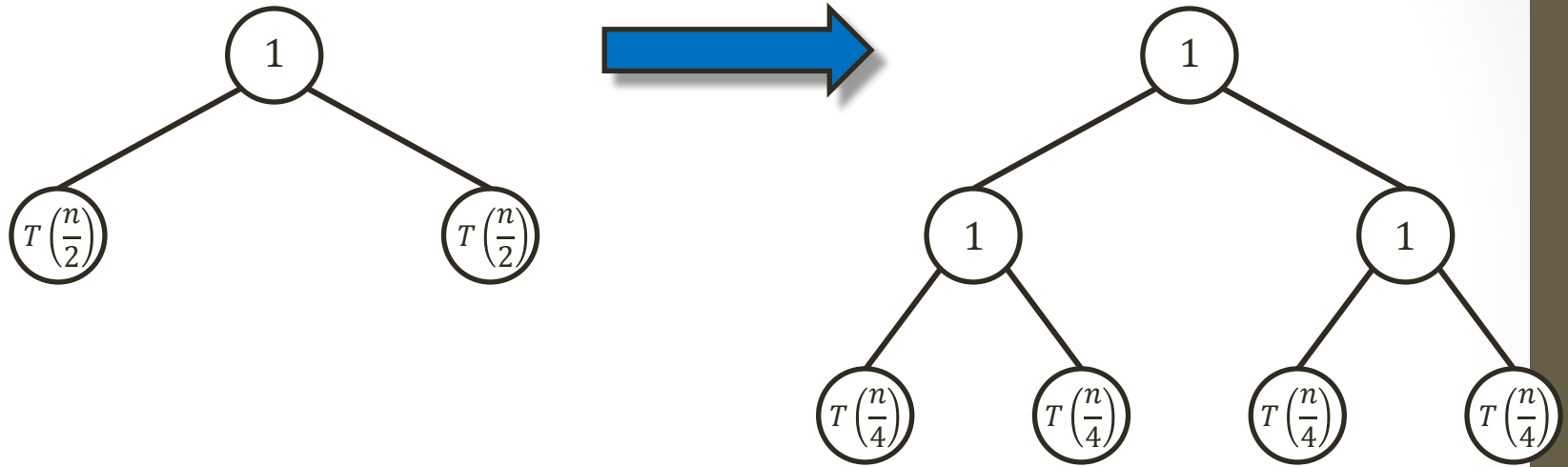
Complexidade de algoritmos

$T(n)$



$$T(n) = 2T(n/2) + 1$$

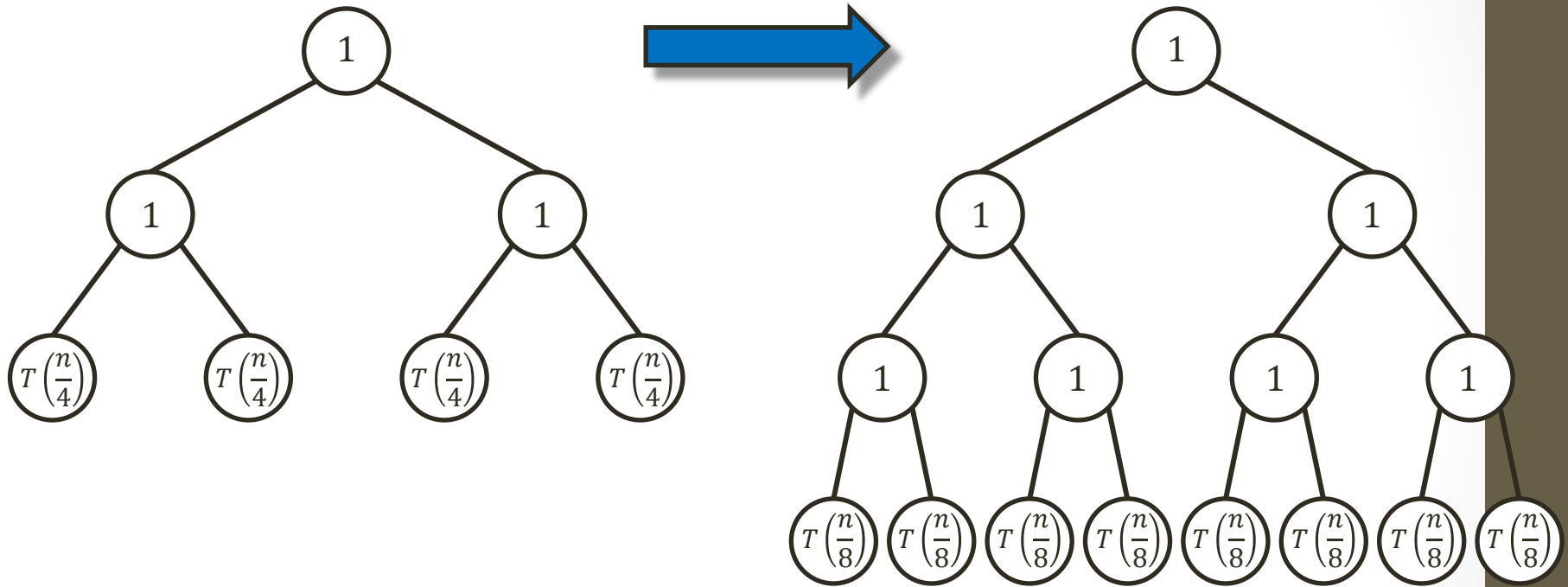
Complexidade de algoritmos



$$T(n) = 2T(n/2) + 1$$

$$T(n/2) = 2T(n/4) + 1$$

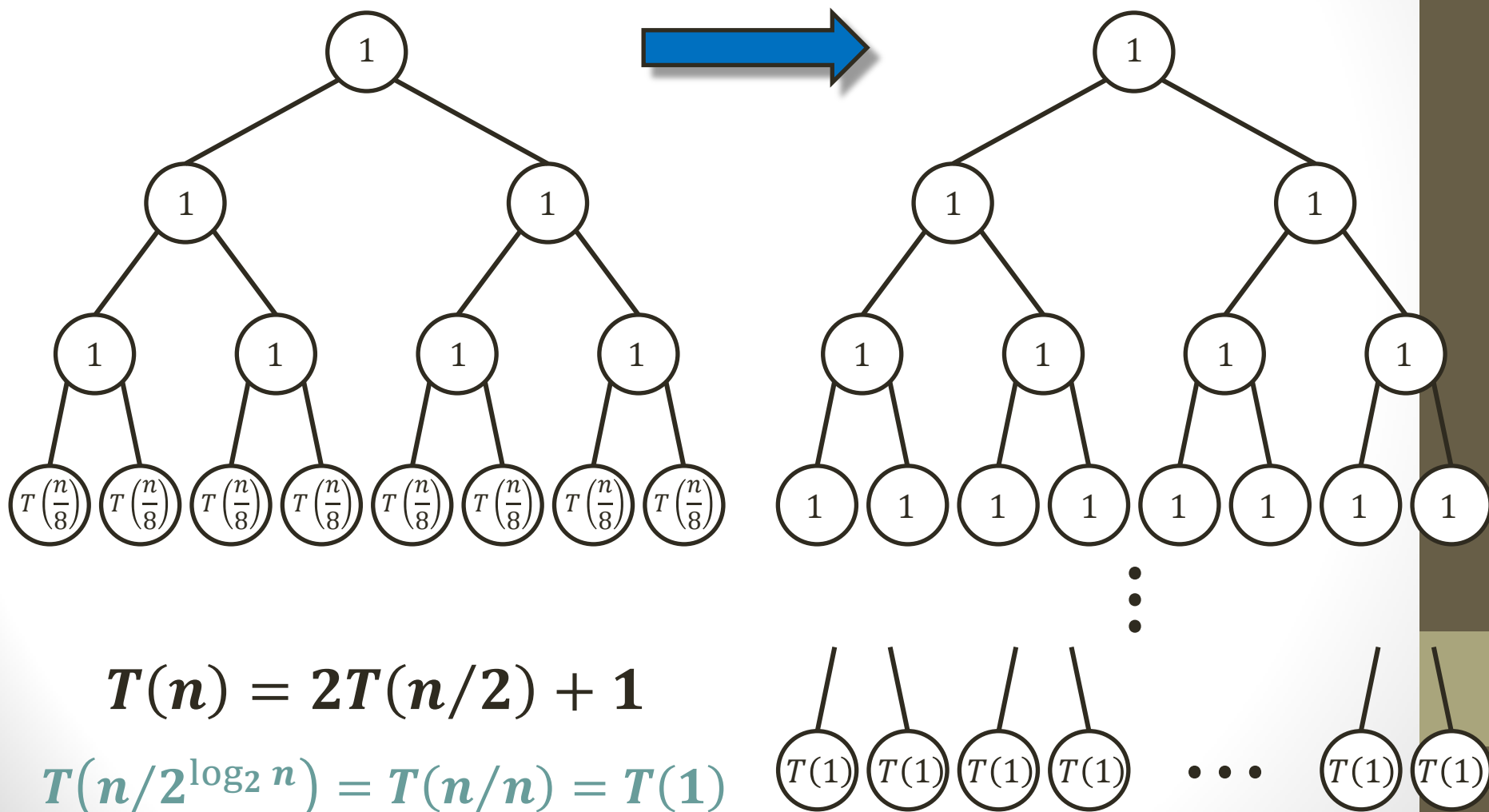
Complexidade de algoritmos



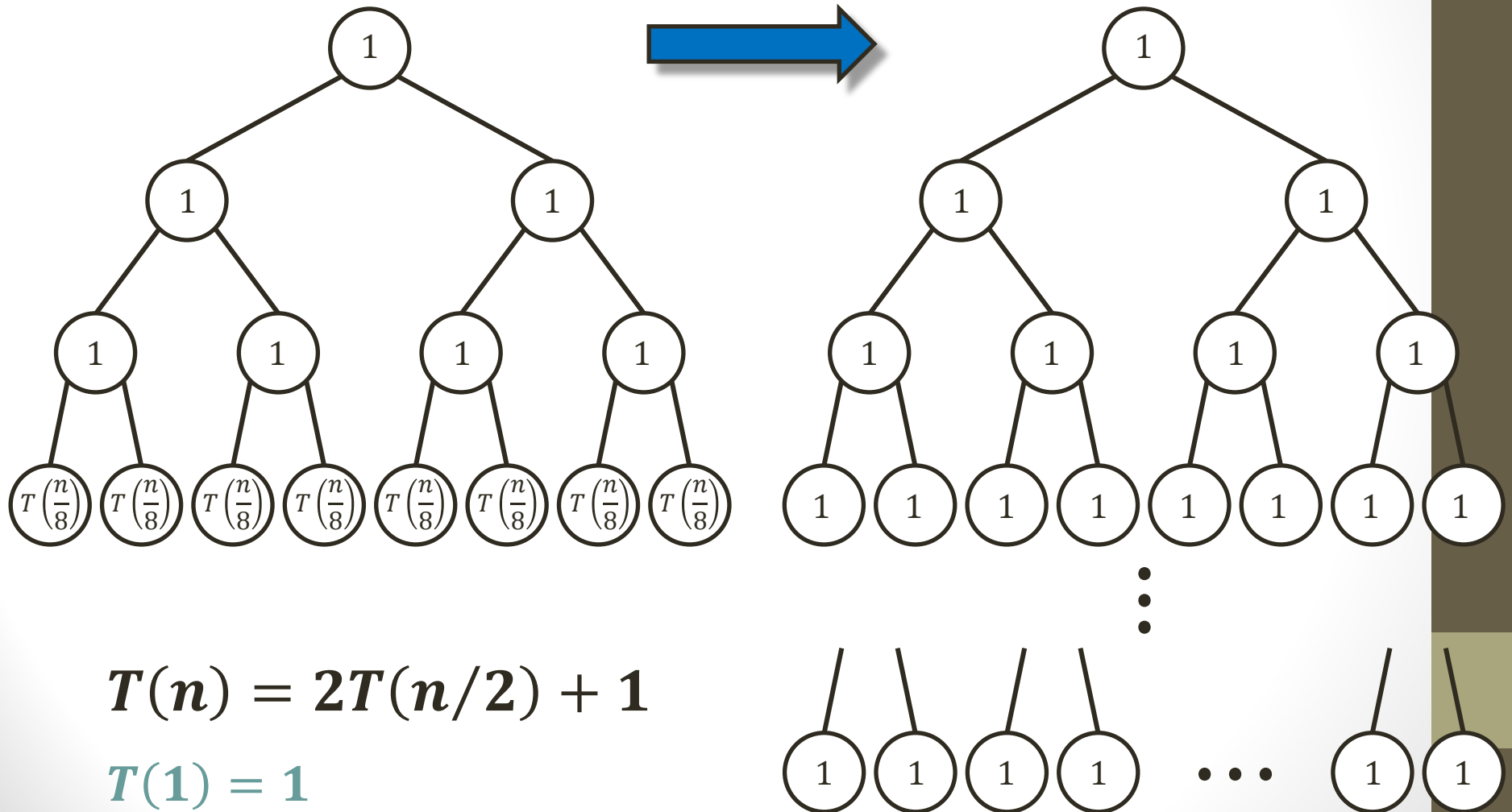
$$T(n) = 2T(n/2) + 1$$

$$T(n/4) = 2T(n/8) + 1$$

Complexidade de algoritmos

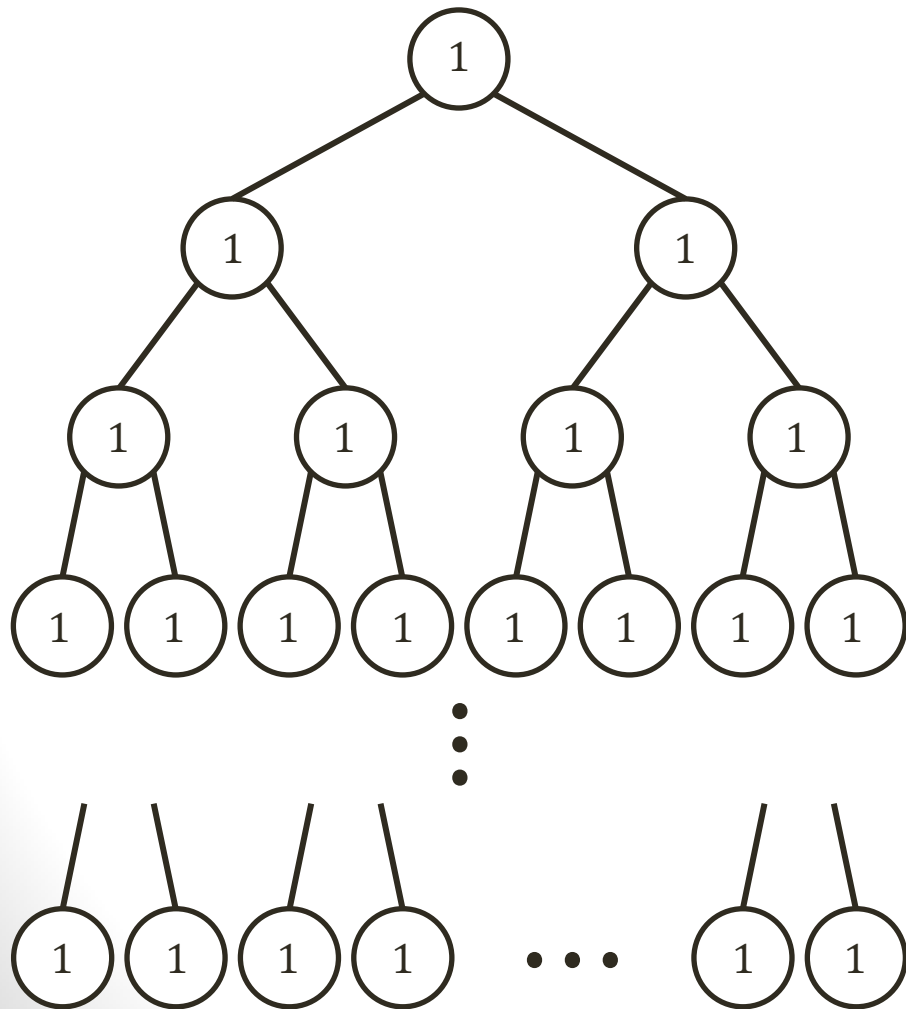


Complexidade de algoritmos

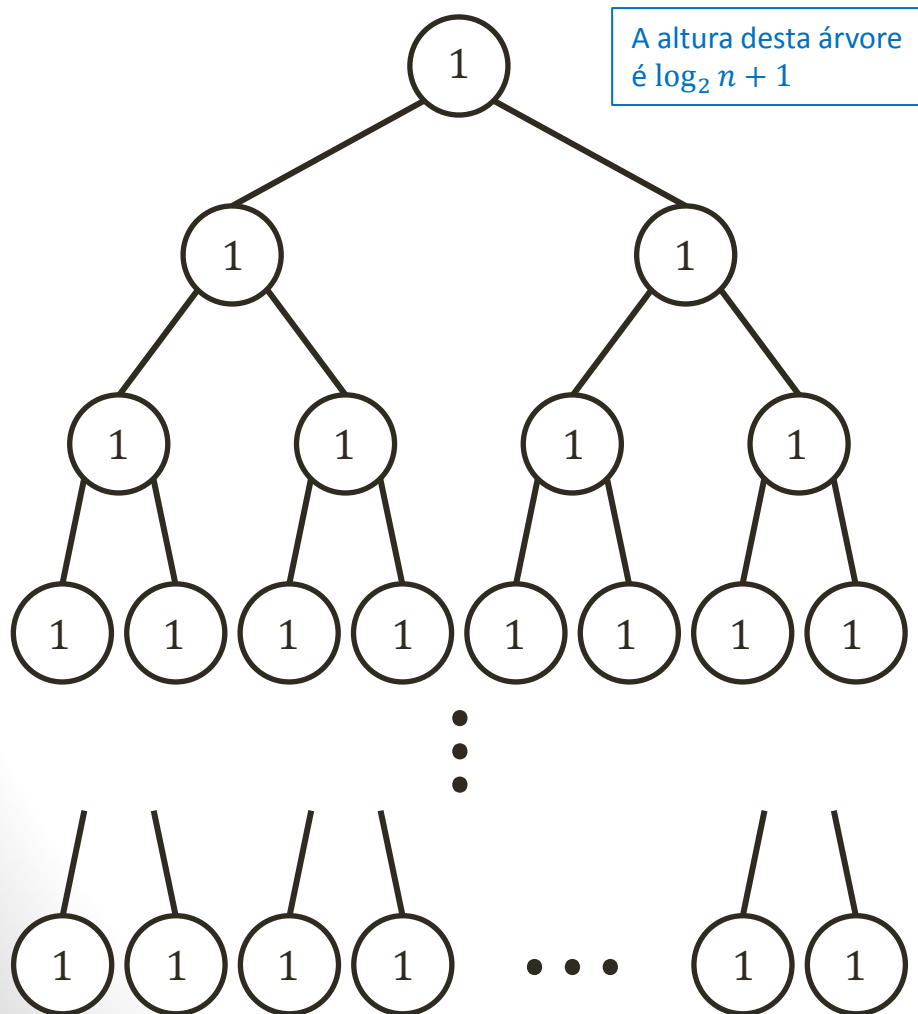


Complexidade de algoritmos

$$T(n) = 2T(n/2) + 1$$



Complexidade de algoritmos



$$T(n) = 2T(n/2) + 1$$

$$T(n) = 2^0 + 2^1 + 2^2 + \dots + 2^{\log_2 n}$$

$$S_n = a_1 \cdot \frac{1 - q^n}{1 - q} \quad (\text{Soma dos termos de uma P.G. finita})$$

$$T(n) = 1 \cdot \frac{1 - 2^{(\log_2 n + 1)}}{1 - 2}$$

$$T(n) = \frac{1 - 2 \cdot 2^{\log_2 n}}{1 - 2}$$

$$T(n) = \frac{1 - 2n}{1 - 2}$$

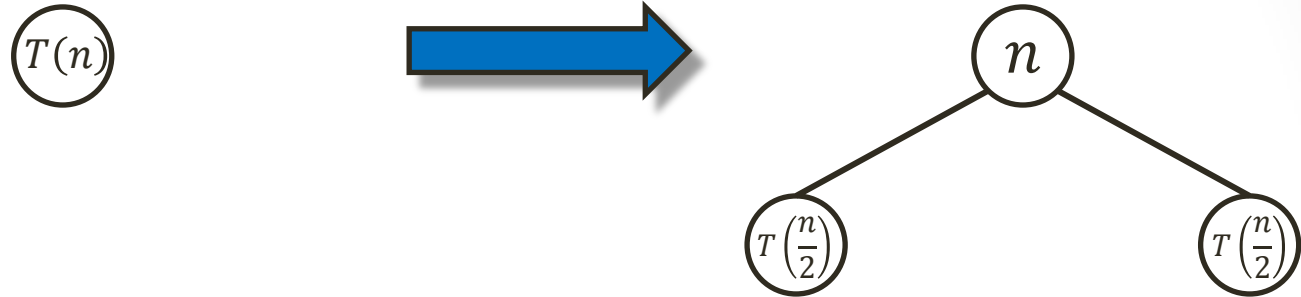
$$T(n) = 2n - 1$$

$$T(n) = O(n)$$

Complexidade de algoritmos

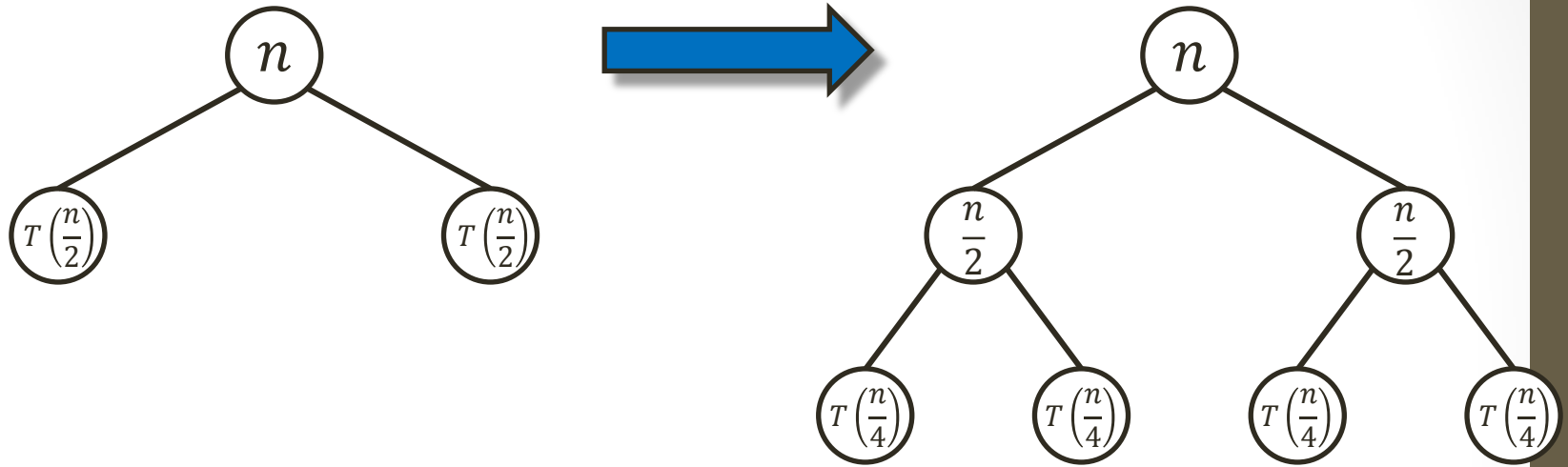
$$T(n) = 2T(n/2) + n \quad \longrightarrow \quad \textcircled{T(n)}$$

Complexidade de algoritmos



$$T(n) = 2T(n/2) + n$$

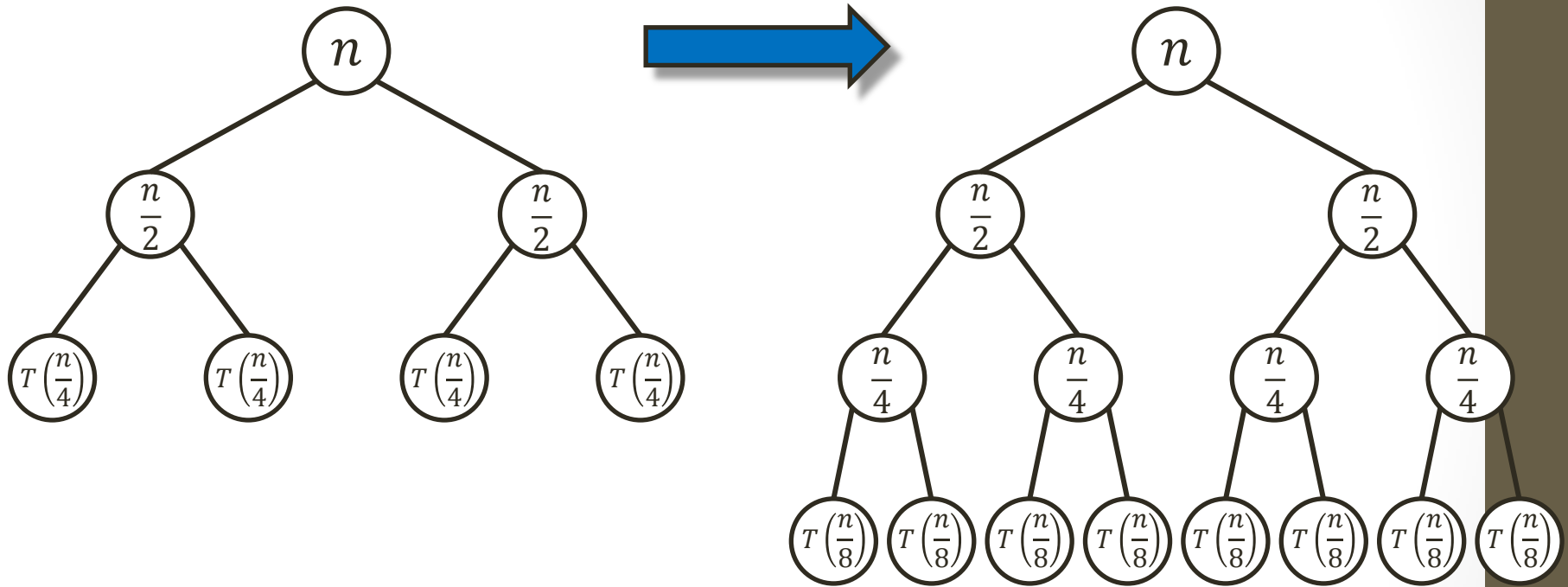
Complexidade de algoritmos



$$T(n) = 2T(n/2) + n$$

$$T(n/2) = 2T(n/4) + n/2$$

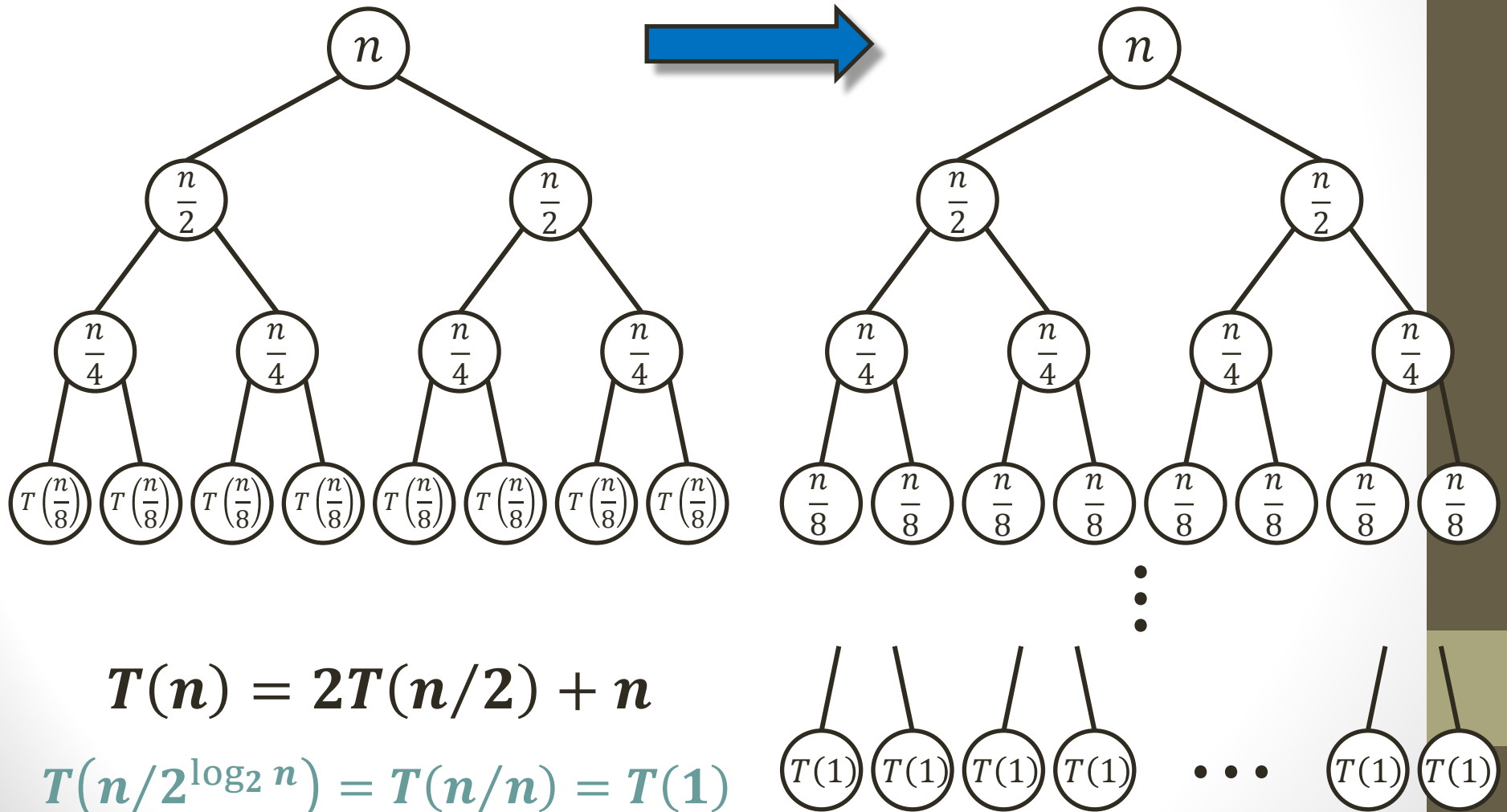
Complexidade de algoritmos



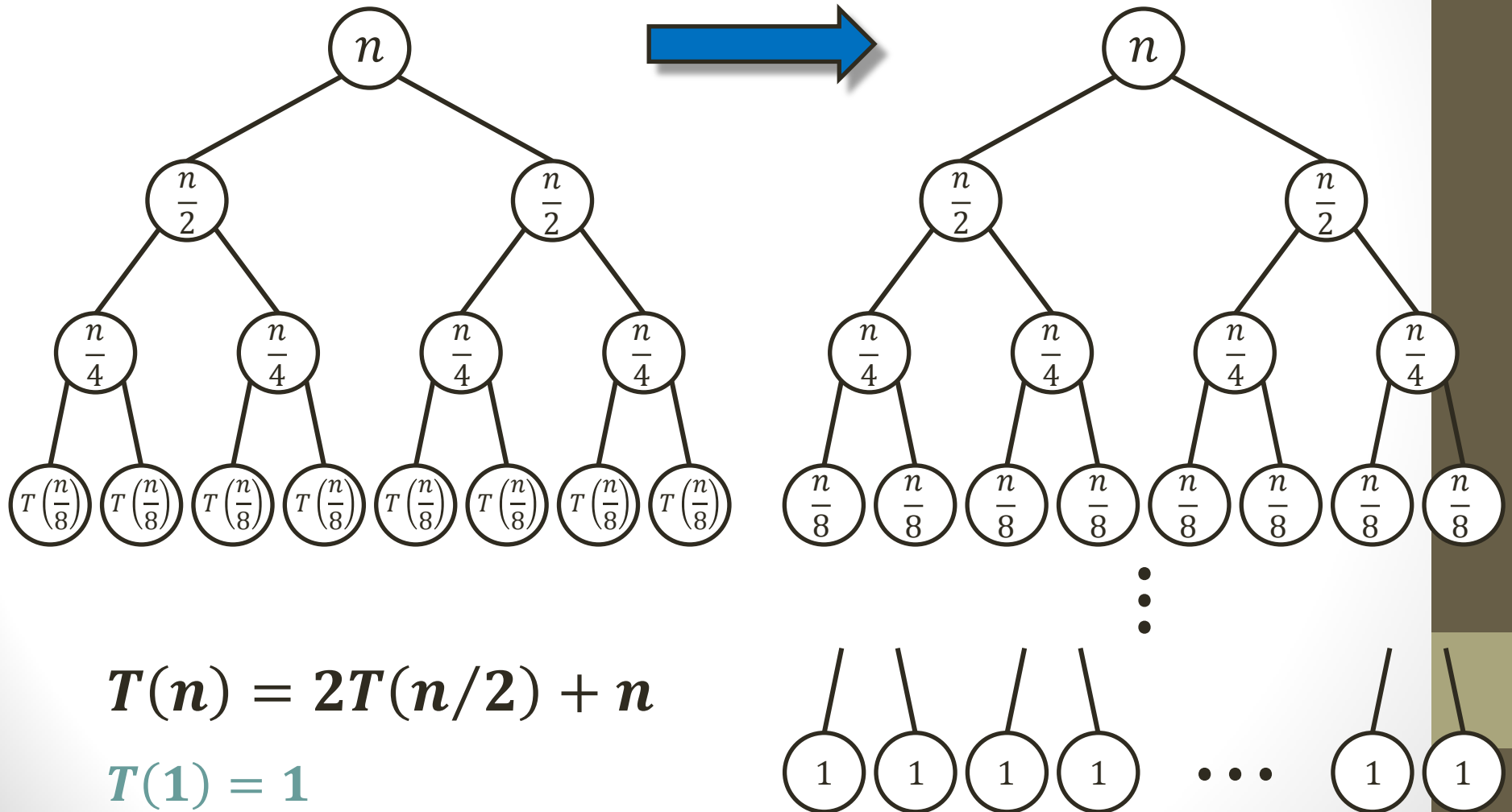
$$T(n) = 2T(n/2) + n$$

$$T(n/4) = 2T(n/8) + n/4$$

Complexidade de algoritmos

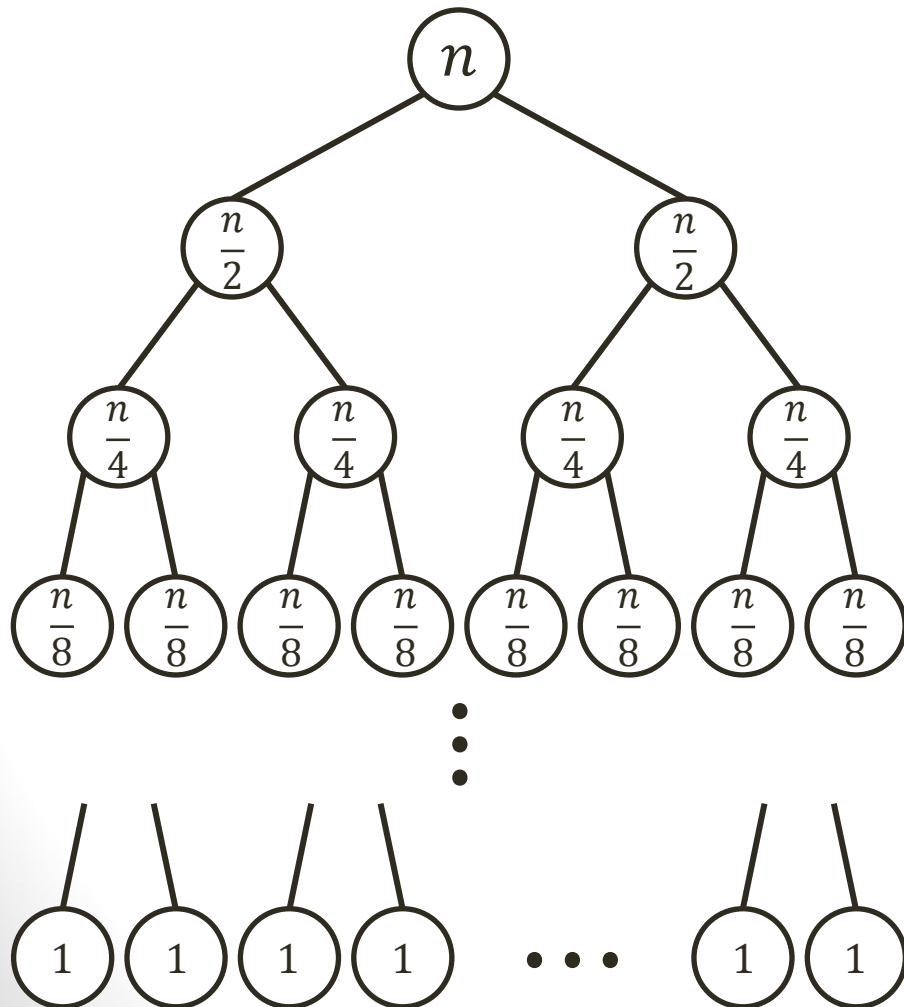


Complexidade de algoritmos

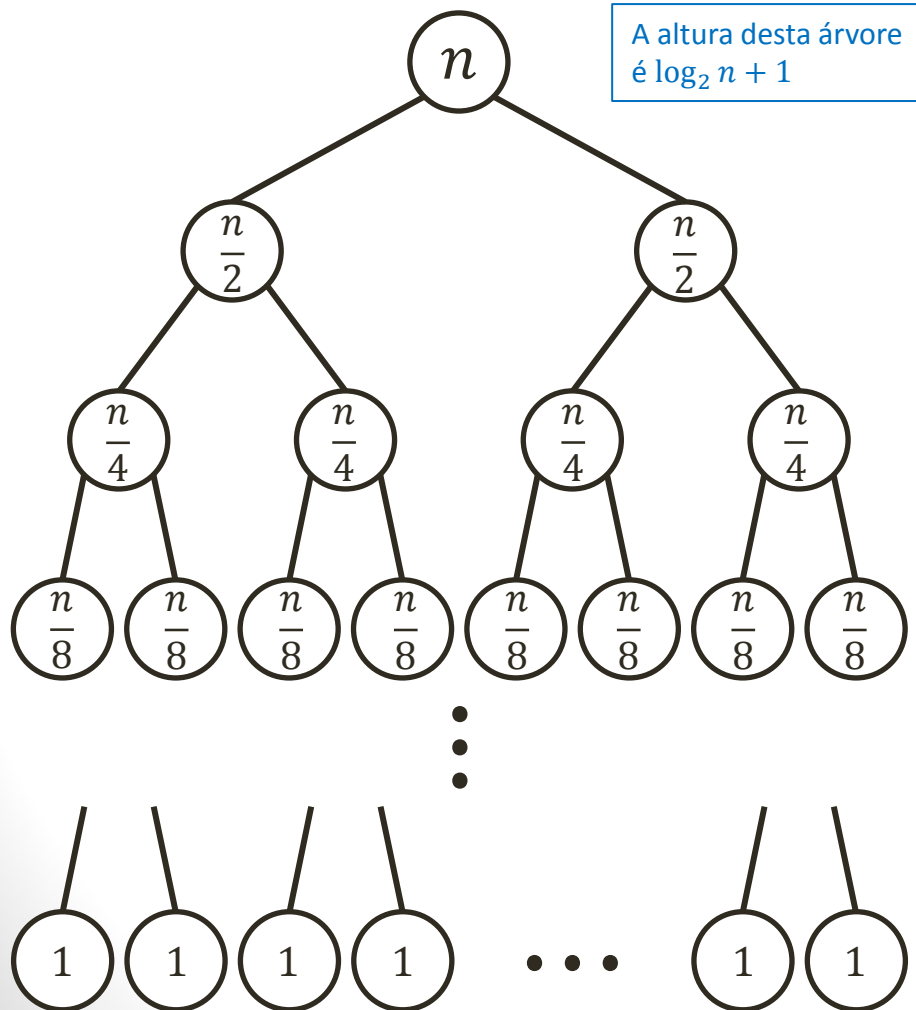


Complexidade de algoritmos

$$T(n) = 2T(n/2) + n$$



Complexidade de algoritmos



$$T(n) = 2T(n/2) + n$$

(Note que a soma dos custos em cada nível da árvore é n)

$$T(n) = n \cdot (\text{altura da árvore})$$

$$T(n) = n \cdot (\log_2 n + 1)$$

$$T(n) = n \log_2 n + n$$

$$T(n) = O(n \log_2 n)$$

Formas de Alocação

- Alocação estática × Alocação dinâmica
- Alocação estática
 - A alocação estática de memória ocorre antes do algoritmo ser executado
- Alocação dinâmica
 - A alocação dinâmica de memória ocorre durante a execução do algoritmo

Formas de Alocação

- Alocação estática
 - A alocação estática de memória ocorre antes do algoritmo ser executado
 - Em programas, a memória estática é alocada em tempo de compilação
 - O tamanho da memória alocada estaticamente é fixo e previamente definido
 - Variáveis estáticas são variáveis cuja memória foi alocada estaticamente.
 - O tempo de vida de uma variável estática dura até o fim da execução do algoritmo
 - A memória que foi alocada estaticamente é liberada automaticamente ao final da execução do algoritmo

Formas de Alocação

- Alocação dinâmica
 - A alocação dinâmica de memória ocorre durante a execução do algoritmo
 - Em programas, a memória dinâmica é alocada em tempo de execução
 - O tamanho da memória alocada dinamicamente é variável e definido no algoritmo
 - Variáveis dinâmicas são variáveis cuja memória foi alocada dinamicamente.
 - O tempo de vida de uma variável dinâmica é controlado pelo algoritmo
 - O algoritmo deve liberar a memória que foi alocada dinamicamente

Formas de Alocação

- Alocação dinâmica
- Ponteiros
- Um ponteiro é uma variável que referencia outra variável que foi alocada dinamicamente
- O ponteiro armazena um endereço de memória onde a variável dinâmica foi alocada
- Operações associadas ao tipo de dado ponteiro permitem a alocação e a liberação de memória
- Estudaremos o uso de ponteiros em breve

Formas de Alocação

- Alocação estática × Alocação dinâmica

objetos: 

 a : inteiro;

 P : ^inteiro;

início:

 a ← 5;

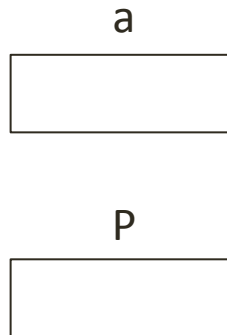
novo(P);

 P^ ← 8;

dispose(P);

fim.

Variáveis estáticas



Endereço (hex)	Valor
:	...
A32Bh	...
A32Ch	...
A32Dh	...
A32Eh	...
A32Fh	...
A330h	...
A331h	...
A332h	...
:	...

Formas de Alocação

- Alocação estática × Alocação dinâmica

objetos:

a : inteiro;

P : ^inteiro;

início:

a ← 5;

novo(P);

P^ ← 8;

dispose(P);

fim.

Variáveis estáticas

a

5

P

Endereço (hex)	Valor
⋮	...
A32Bh	...
A32Ch	...
A32Dh	...
A32Eh	...
A32Fh	...
A330h	...
A331h	...
A332h	...
⋮	...

Formas de Alocação

- Alocação estática × Alocação dinâmica

objetos:

a : inteiro;

P : ^inteiro;

início:

a ← 5;

novo(P);

P^ ← 8;

dispose(P);

fim.

Variáveis estáticas

a

5

P

A32Eh

Endereço (hex)	Valor
⋮	...
A32Bh	...
A32Ch	...
A32Dh	...
A32Eh	...
A32Fh	...
A330h	...
A331h	...
A332h	...
⋮	...

Formas de Alocação

- Alocação estática × Alocação dinâmica

objetos:

a : inteiro;

P : ^inteiro;

início:

a ← 5;

novo(P);

P^ ← 8;

dispose(P);

fim.

Variáveis estáticas

a

5

P

A32Eh

Endereço (hex)	Valor
⋮	...
A32Bh	...
A32Ch	...
A32Dh	...
A32Eh	08h
A32Fh	00h
A330h	...
A331h	...
A332h	...
⋮	...

Formas de Alocação

- Alocação estática × Alocação dinâmica

objetos:

a : inteiro;

P : ^inteiro;

início:

a ← 5;

novo(P);

P^ ← 8;

dispose(P);

fim.

Variáveis estáticas

a

5

P

A32Eh

Endereço (hex)	Valor
:	...
A32Bh	...
A32Ch	...
A32Dh	...
A32Eh	...
A32Fh	...
A330h	...
A331h	...
A332h	...
:	...

Formas de Alocação

- Alocação estática × Alocação dinâmica

objetos: 

 a : inteiro;

 P : ^inteiro;

início:

 a ← 5;

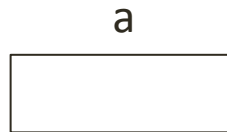
novo(P);

 P^ ← 8;

dispose(P);

fim.

Variáveis estáticas



P

Formas de Alocação

- Alocação estática × Alocação dinâmica

objetos:

a : inteiro;

P : ^inteiro;

início:

a ← 5;

novo(P);

P^ ← 8;

dispose(P);

fim.

Variáveis estáticas

a

5

P



Formas de Alocação

- Alocação estática × Alocação dinâmica

objetos:

a : inteiro;

P : ^inteiro;

início:

a ← 5;

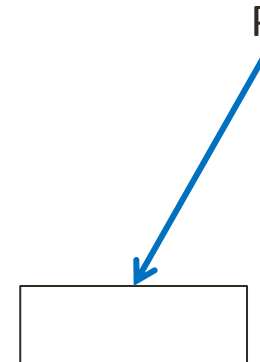
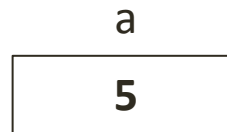
novo(P);

P^ ← 8;

dispose(P);

fim.

Variáveis estáticas



Formas de Alocação

- Alocação estática × Alocação dinâmica

objetos:

a : inteiro;

P : ^inteiro;

início:

a ← 5;

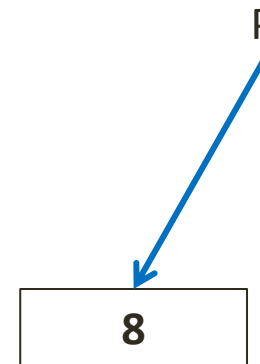
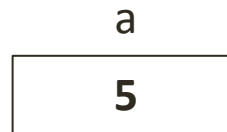
novo(P);

P^ ← 8;

dispose(P);

fim.

Variáveis estáticas



Formas de Alocação

- Alocação estática × Alocação dinâmica

objetos:

a : inteiro;

P : ^inteiro;

início:

a ← 5;

novo(P);

P^ ← 8;

dispose(P);

fim.

Variáveis estáticas

a



P

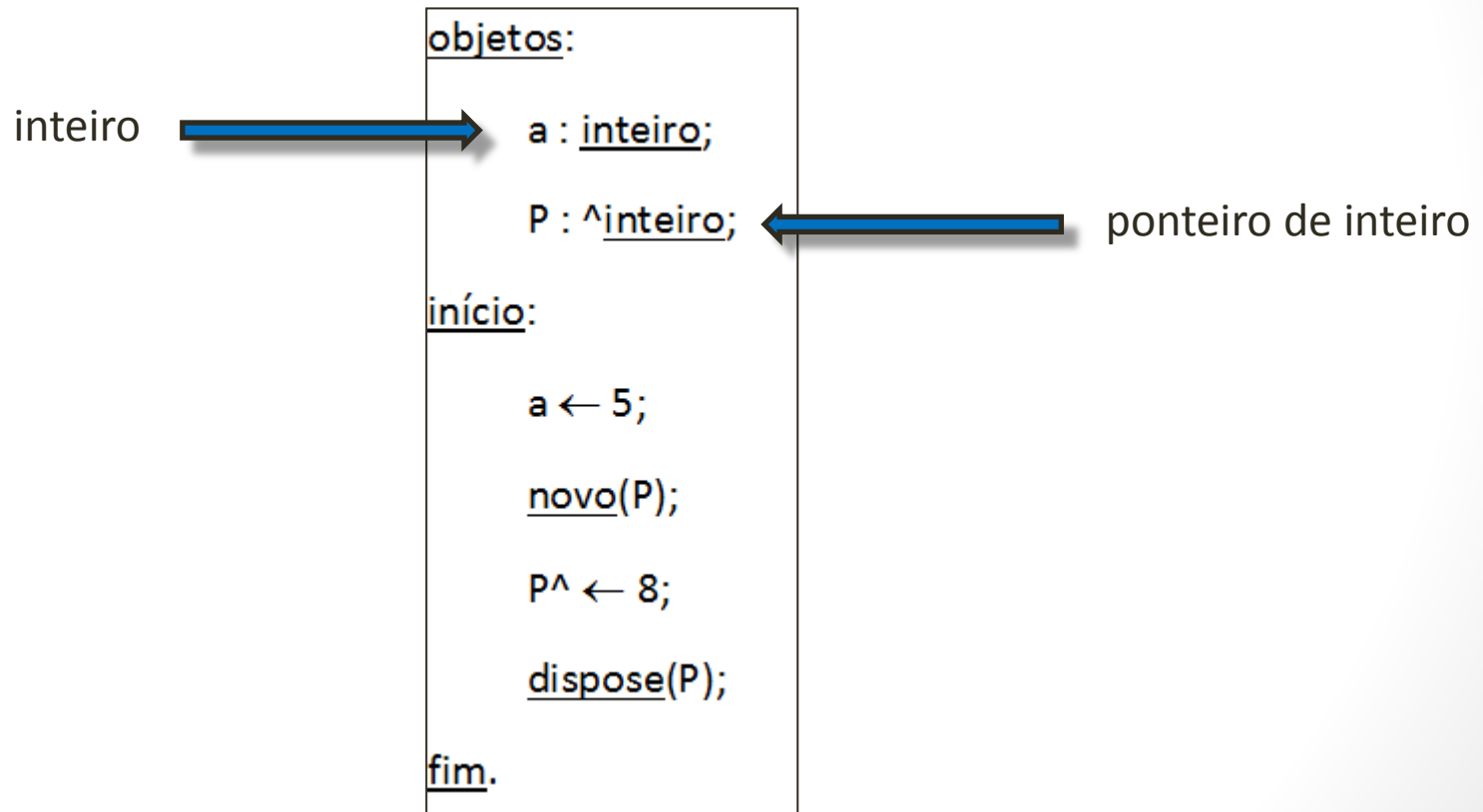


Formas de Alocação

- Ponteiros
- Um ponteiro é uma variável que referencia outra variável que foi alocada dinamicamente
- O ponteiro armazena um endereço de memória onde a variável dinâmica foi alocada
- Operações associadas ao tipo de dado ponteiro permitem a alocação e a liberação de memória
- Ponteiros são usualmente associados a tipos de dados. Esta associação permite atribuir um tipo à variável alocada dinamicamente, criando uma relação com o tamanho do espaço alocado e com as operações que podem ser executadas com o dado armazenado

Formas de Alocação

- Ponteiros



Formas de Alocação

- Operações com ponteiros
- novo(P)
- Atribuição
- Derreferenciação
- Operações com a variável dinâmica
- nil
- dispose(P)

Formas de Alocação

- Operações com ponteiros
- novo(P)
- Cria uma variável dinamicamente e faz o ponteiro P apontar para ela
- Alocação de memória
- Tipo de dado associado ao ponteiro define a quantidade de memória alocada e as operações que podem realizadas na variável dinamicamente alocada
- ex: malloc (C) e new (Pascal)

Formas de Alocação

- Operações com ponteiros : novo(P)

objetos:

P : ^inteiro;

início:

⋮

novo(P);

$P^{\wedge} \leftarrow 8$;

novo(P);

⋮

fim.

Formas de Alocação

- Operações com ponteiros
- Atribuição: $\leftarrow$
- $P \leftarrow Q;$
- Faz o ponteiro P apontar para onde o ponteiro Q está apontando
- Ou seja, faz o ponteiro P apontar para a mesmo endereço de memória apontado por Q

Formas de Alocação

- Operações com ponteiros : Atribuição: ←

objetos:

P,Q : inteiro;

início:

⋮

novo(P);

$P^{\wedge} \leftarrow 8;$

$Q \leftarrow P;$

escreva($Q^{\wedge}$);

novo(P);

⋮

fim.

Formas de Alocação

- Operações com ponteiros : Derreferenciação
- Faz referência à variável dinâmica apontada pelo ponteiro
- P : ponteiro
- $P^{\wedge}$: a variável apontada pelo ponteiro P
- ex: P^* (C) e $P^{\wedge}$ (Pascal)

Formas de Alocação

- Operações com ponteiros : Derreferenciação

objetos:

P : ^inteiro;

início:

⋮

novo(P);

$P^{\wedge} \leftarrow 10$;

escreva($P^{\wedge}$);

⋮

fim.

Formas de Alocação

- Operações com ponteiros
- Operações com a variável dinâmica
- As mesmas operações utilizadas em variáveis estáticas, desde que os ponteiros estejam derreferenciados
- As operações com a variável dinâmica dependem do tipo de dado associado ao ponteiro em sua declaração
- ex: operações associadas ao tipo inteiro podem ser aplicadas em variáveis dinâmicas apontadas por um ponteiro do tipo ponteiro de inteiro

Formas de Alocação

- Operações com ponteiros : Operações com a variável dinâmica

objetos:

P : ^inteiro;

a : inteiro;

início:

:

novo(P);

P[^] ← 5;

a ← 4;

P[^] ← P[^] + a + 3;

escreva(P[^]);

:

fim.

Formas de Alocação

- Operações com ponteiros
- nil
- É uma constante nula para a qual um ponteiro pode ser apontado
- É uma maneira segura de indicar que o ponteiro não aponta para algum objeto válido
- Dizemos que um ponteiro está aterrado ($\equiv$) quando aponta para nil
- Ponteiro apontando para nil $\neq$ ponteiro não inicializado
- ex: null (C) e nil (Pascal)

Formas de Alocação

- Operações com ponteiros : nil

objetos:

P,Q : ^inteiro;

início:

:

Q $\leftarrow$ nil;

novo(P);

Q $\leftarrow$ P;

P $\leftarrow$ nil;

:

fim.

Formas de Alocação

- Operações com ponteiros
- dispose(P)
- Libera a memória associada à variável dinâmica apontada por P
- Devolve o espaço de memória liberado à memória livre disponível
- ex : free (C) e dispose (Pascal)

Formas de Alocação

- Operações com ponteiros : dispose(P)

objetos:

P,Q : ^inteiro;

início:

⋮

novo(P);

Q ← P;

dispose(Q);

⋮

fim.

Formas de Alocação

- Registro

tipo TpReg = registro

início

campo_1 : tipo_1;

campo_2 : tipo_2;

campo_3 : tipo_3;

⋮

campo_n : tipo_n;

fim;

Formas de Alocação

- Registro

tipo TpReg = registro
início

campo_1 : tipo_1;
campo_2 : tipo_2;
campo_3 : tipo_3;
:
campo_n : tipo_n;

fim;

- Em C :

typedef struct TpReg
{
 tipo_1 campo_1;
 tipo_2 campo_2;
 tipo_3 campo_3;
 :
 tipo_n campo_n;
};

Formas de Alocação

- Registro

tipo TpReg = registro
início

campo_1 : tipo_1;
campo_2 : tipo_2;
campo_3 : tipo_3;
:
campo_n : tipo_n;

fim;

- Em Pascal :

type TpReg = record
begin

campo_1 : tipo_1;
campo_2 : tipo_2;
campo_3 : tipo_3;
:
campo_n : tipo_n;

end;

Formas de Alocação

- Registro

tipo TpNó = registro
início



info : tipo;

prox : ^TpNó;

fim;

onde:

- info : campo de informação (pode ter um ou mais campos)
- prox : é um ponteiro do tipo TpNó

Formas de Alocação

- Registro

Exemplo(P);

P ← nil;

leia(a);

enquanto a > 0

faça

| novo(Q);

| Q^.info ← a;

| Q^.prox ← P;

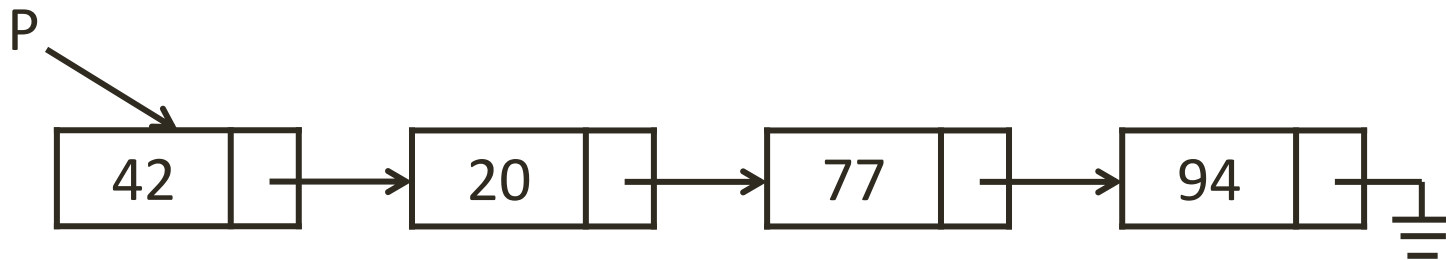
| P ← Q;

| leia(a);

(P e Q são duas variáveis do tipo **ponteiro de TpNó**)

Formas de Alocação

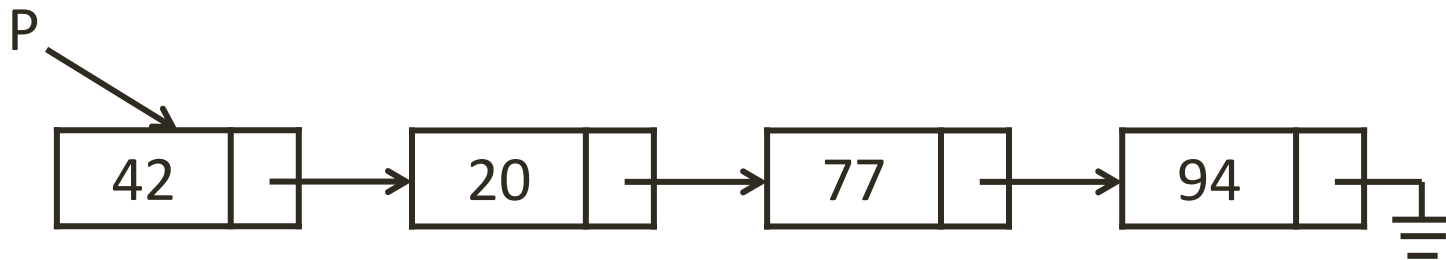
- Lista ligada (ou lista encadeada)



- Cada elemento (variável dinâmica) desta lista contém um (ou mais) campo(s) de informação e um ponteiro para o elemento seguinte da lista
- O acesso ao primeiro elemento requer o uso de um ponteiro
- O ponteiro do último elemento da lista ligada está **aterrado** (isto é, aponta para nil)

Formas de Alocação

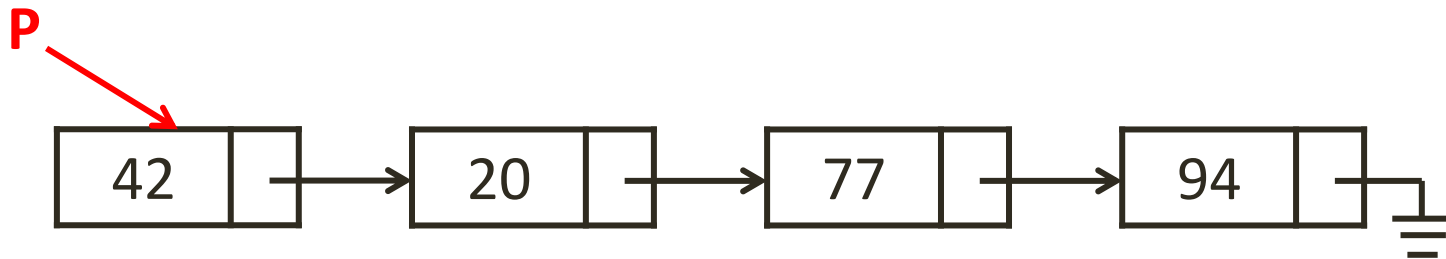
- Lista ligada (ou lista encadeada)



- O exemplo acima é de uma lista **simplesmente** ligada e sem **cabeça**. Será o tipo de lista ligada que mais utilizaremos nesta disciplina
- Veremos listas duplamente ligadas adiante na disciplina
- Posteriormente, também veremos listas ligadas com cabeça de lista

Formas de Alocação

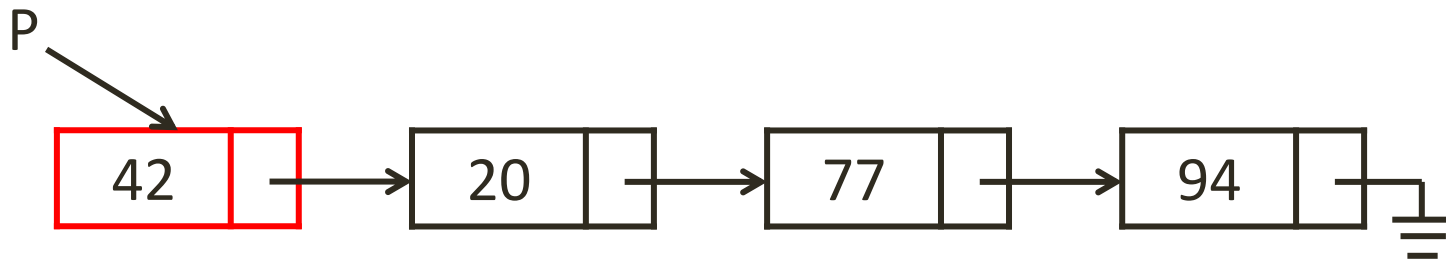
- Lista ligada (ou lista encadeada)



- **P**
- (ponteiro P)
- Ponteiro de registro

Formas de Alocação

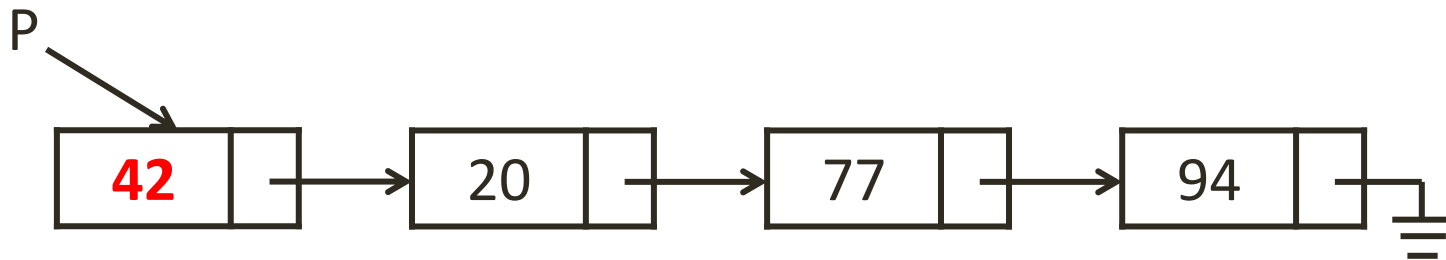
- Lista ligada (ou lista encadeada)



- $P^{\wedge}$
- (a variável dinâmica apontada pelo ponteiro P)
- Registro

Formas de Alocação

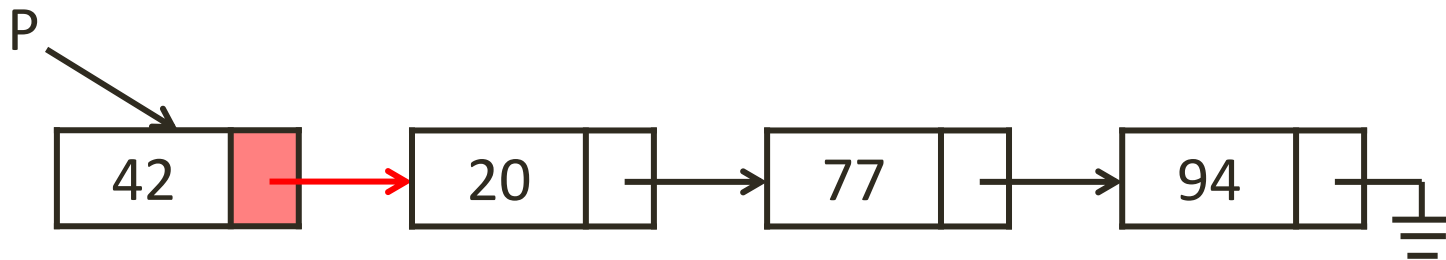
- Lista ligada (ou lista encadeada)



- **P^.info**
- (o valor do campo **info** da variável dinâmica P^)
- Variável do tipo escolhido pelo usuário para armazenar o dado. Pode ser tipo simples ou estruturado

Formas de Alocação

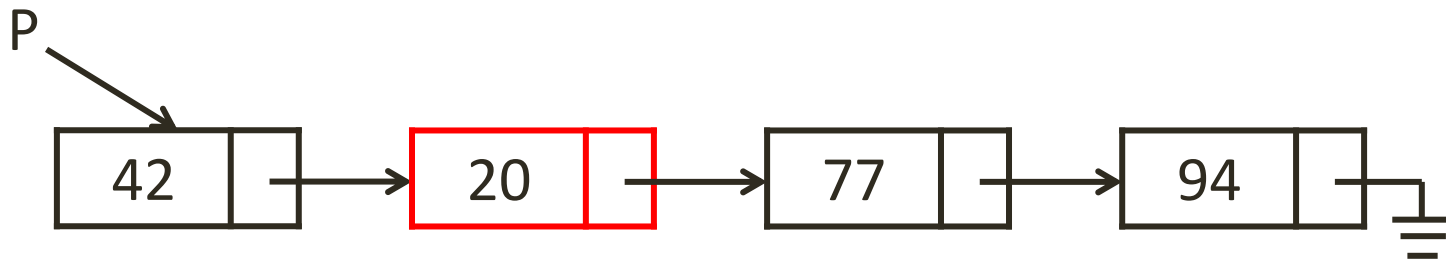
- Lista ligada (ou lista encadeada)



- **$P^{\wedge}.\text{prox}$**
- (O campo **prox** da variável dinâmica $P^{\wedge}$ é um ponteiro que aponta para o elemento seguinte da lista ligada)
- ($P^{\wedge}.\text{prox}$ estará aterrado se $P^{\wedge}$ for o último elemento da lista ligada)
- Ponteiro de registro

Formas de Alocação

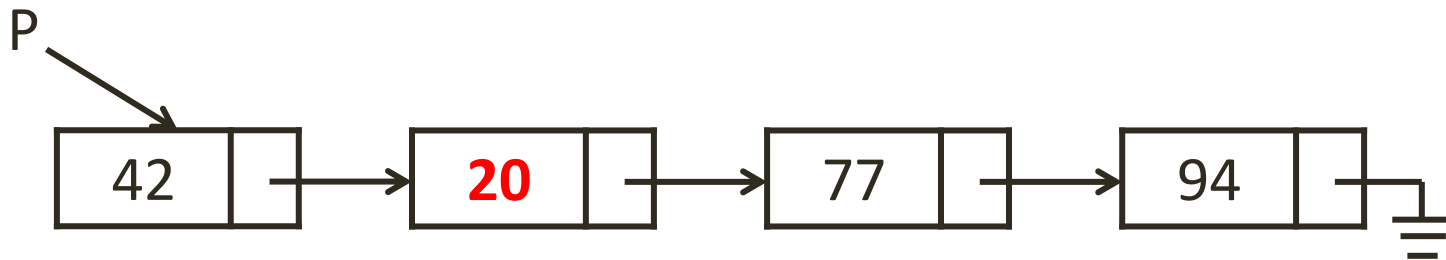
- Lista ligada (ou lista encadeada)



- **$P^{\wedge}.prox^{\wedge}$**
- (a variável dinâmica apontada pelo ponteiro $P^{\wedge}.prox$)
- Registro
- **OBS** : se $P^{\wedge}.prox$ estiver aterrado, não haverá $P^{\wedge}.prox^{\wedge}$ (**ERRO**)

Formas de Alocação

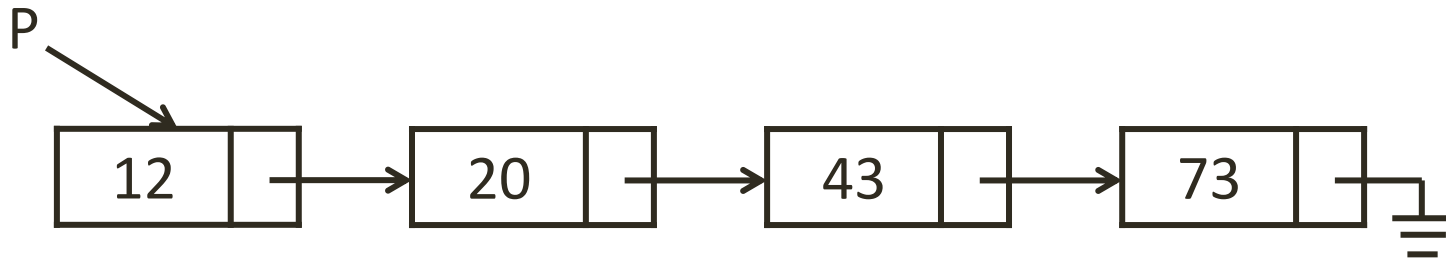
- Lista ligada (ou lista encadeada)

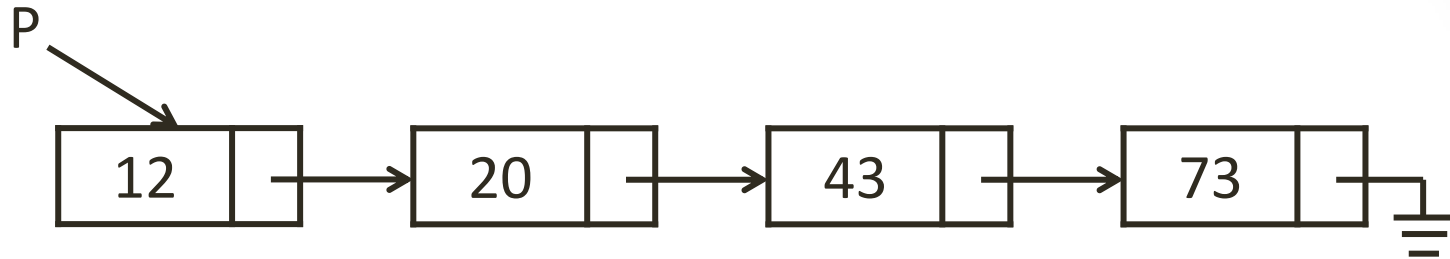


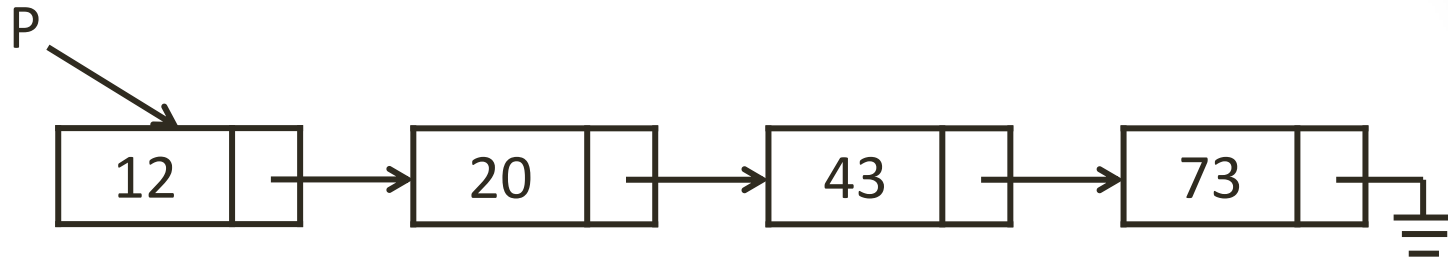
- **P^.prox^.info**
- (o valor do campo **info** da variável dinâmica P^.prox^)
- **OBS** : se P^.prox estiver aterrado, não haverá P^.prox^ (**ERRO**)

Formas de Alocação

- Exercício : faça uma função que receba uma lista ligada P e um elemento x e que verifique se x está nesta lista
- A função deve devolver um ponteiro para a variável dinâmica que contenha x, ou nil se x não estiver na lista P







Busca(P,x);

$Q \leftarrow P;$

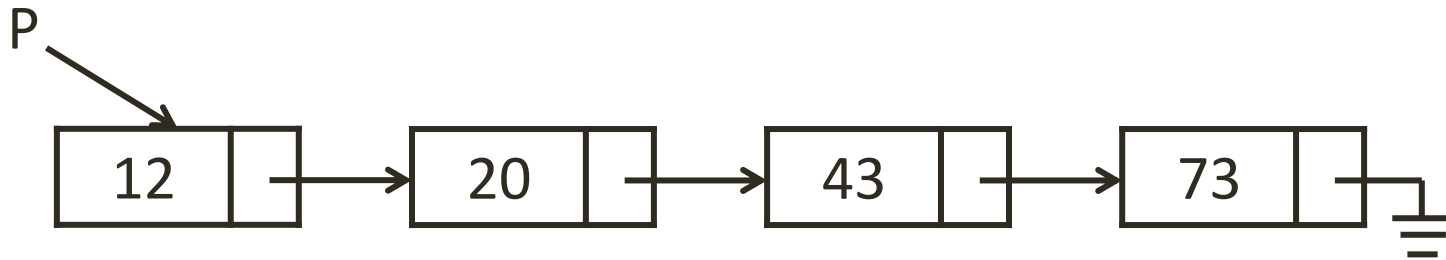
enquanto $(Q \neq \text{nil}) \ \underline{\text{E}} \ (Q^{\wedge}.\text{info} \neq x)$

faça $| \quad Q \leftarrow Q^{\wedge}.\text{prox};$

$\text{Busca} \leftarrow Q;$

Formas de Alocação

- Exercício : faça uma função que receba uma lista ligada P ordenada e um elemento x. Esta função deve inserir o elemento x na lista P
- A lista P deve continuar ordenada após a inserção de x



Formas de Alocação

InserirOrdenado(P, x);

novo(Q);

Q^.info $\leftarrow$ x;

se (P = nil) OU (x $\leq$ P^.info)

então

Q^.prox $\leftarrow$ P;

P $\leftarrow$ Q;

senão

R $\leftarrow$ P;

enquanto (R^.prox $\neq$ nil) E (R^.prox^.info < x)

faça

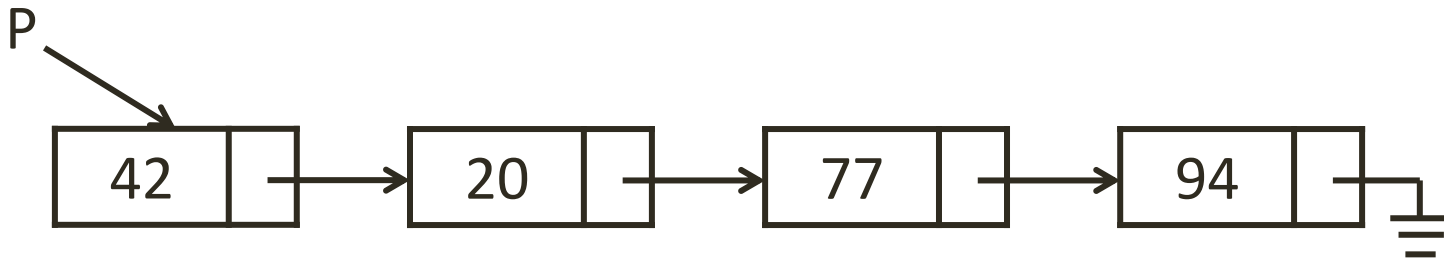
R $\leftarrow$ R^.prox;

Q^.prox $\leftarrow$ R^.prox;

R^.prox $\leftarrow$ Q;

Formas de Alocação

- Exercício : faça uma função que remova um elemento x de uma lista ligada P, caso x esteja nesta lista
- (Assuma que a lista ligada P não tenha elementos repetidos)



Formas de Alocação

RemoveElemento(P, x);

se P $\neq$ nil

então

se P[^].info = x

então

Q $\leftarrow$ P;

P $\leftarrow$ P[^].prox;

dispose(Q);

senão

R $\leftarrow$ P;

enquanto (R[^].prox $\neq$ nil) $\underline{\underline{E}}$ (R[^].prox[^].info $\neq$ x)

faça

R $\leftarrow$ R[^].prox;

se R[^].prox $\neq$ nil

então

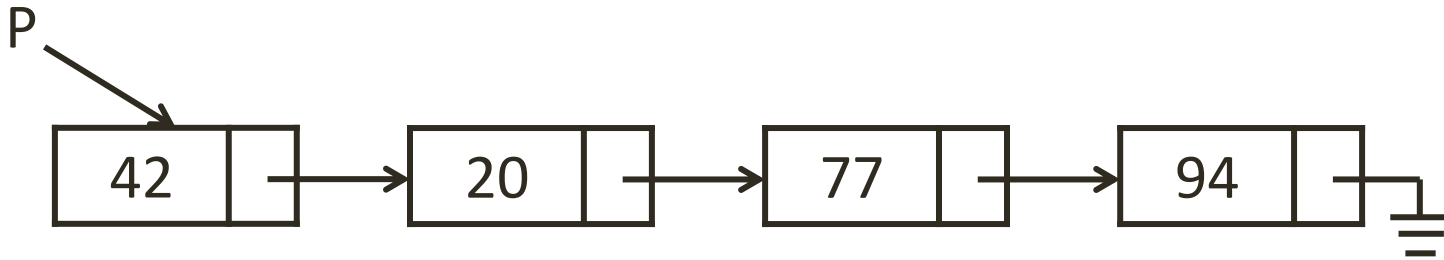
Q $\leftarrow$ R[^].prox;

R[^].prox $\leftarrow$ Q[^].prox;

dispose(Q);

Formas de Alocação

- Vetores × listas ligadas



- Memória
- Acesso a elementos
- Operações

Exercícios

- Recursividade
- Faça uma função que receba um vetor de inteiros e que calcule recursivamente a soma de todos os elementos do vetor
- Faça uma função que receba uma lista ligada que armazena números inteiros e que calcule recursivamente a soma de todos os elementos desta lista ligada

Exercícios

- Recursividade
- Faça uma função que receba um inteiro $n > 0$ e que escreva os n primeiros números da sequência de Fibonacci
 - (a) Escreva uma função iterativa
 - (b) Escreva uma função recursiva
- Faça uma função que receba um vetor de inteiros que recursivamente pesquise se um determinado elemento x está no vetor

Exercícios

- Recursividade
- Faça uma função que receba dois números inteiros x e n e que calcule recursivamente o valor da expressão x^n
- Faça uma função que receba um vetor de inteiros e que encontre recursivamente o maior elemento do vetor

Exercícios

- Listas ligadas
- Faça uma função que receba uma lista ligada e que escreva/imprima seus elementos na ordem inversa
 - (a) Escreva uma função iterativa
 - (b) Escreva uma função recursiva
- Faça uma função que receba uma lista ligada e que inverta seus elementos
 - (a) Criando uma nova lista ligada
 - (b) Transferindo os elementos da lista de entrada para a lista nova. A lista de entrada seria, então, esvaziada

Exercícios

- Listas ligadas
- Faça uma função que receba uma lista ligada arbitrária e que devolva esta lista em ordem crescente
 - (dica: utilize a função `InserirOrdenado`, vista anteriormente)
- Faça uma função que receba uma lista ligada `P` e que devolva `VERDADEIRO` se `P` estiver ordenada em ordem crescente. E que devolva `FALSO`, caso contrário

Exercícios

- Listas ligadas
- Faça uma função que receba uma lista ligada que armazena números inteiros e que remova desta lista todos os números negativos
- Faça uma função que receba uma lista ligada P que armazena números inteiros e que devolva uma outra lista ligada P2 que contenha os elementos de P de modo que todos os números pares de P antecedam todos os números ímpares de P na nova lista

Exercícios

- Listas ligadas
- Faça uma função que receba uma lista ligada P e que verifique se esta lista contém elementos repetidos
- Faça uma função que receba duas listas ligadas P1 e P2 e que realize a concatenação destas duas listas
 - (A lista ligada P2 deve ser concatenada no final da lista ligada P1)

Exercícios

- Listas ligadas
- Faça uma função que receba duas listas ligadas P1 e P2 ordenadas em ordem crescente e que gere uma única lista ligada ordenada P3
 - (a) Faça uma função sem destruir as listas ligadas P1 e P2. Alocando os novos elementos para P3 e armazenando nesta os elementos de P1 e P2
 - (b) Faça uma função sem a utilização das funções novo e dispose. Somente transferindo os elementos de P1 e P2 para a lista P3. Ao final da função, P1 e P2 serão duas listas vazias

Bibliografia

- ZIVIANI, N.; *Projeto de Algoritmos com Implementações em Pascal e C*, 3ª Edição, Livraria Pioneira Editora, 1996.
- CORMEN, T. H. et al – *Algoritmos* , Trad. Segunda Edição Americana. Editora Campus, 2002.
- WIRTH, N. – *Algoritmos e Estruturas de Dados*. Editora Prentice-Hall, 1989.